SEMANA 13 · ARQUITECTURAS DESACOPLADAS: MENSAJERÍA, COLAS, PUB/SUB

Del acoplamiento a la comunicación asíncrona

Parte 1 de 4 · Unidad 3

Ing. Rodolfo Cañas Cervantes

Universidad de la Costa (CUC) · Ingeniería de Sistemas

Semana 13 · Periodo 2026-2 · Barranquilla, Colombia

Por qué las llamadas en cadena se rompen y cómo un mensaje intermedio devuelve la resiliencia.

Acoplamiento: cuánto sabe un componente de otro

Cuanto menos necesita saber A sobre B, más libre es cada uno para fallar, escalar y cambiar.

Acoplamiento fuerte → **Acoplamiento débil**

Monolito



misma memoria y misma base

Lo que A sabe de B

A conoce el código de B

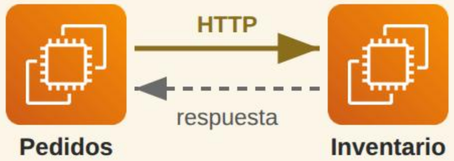
Se compilan, prueban y despliegan siempre juntos

Tiempo: juntos · **Lugar: conocido**

Menos flexible

a cambios y a la escala

Llamada síncrona



A espera mientras B responde

Lo que A sabe de B

A conoce la dirección de B

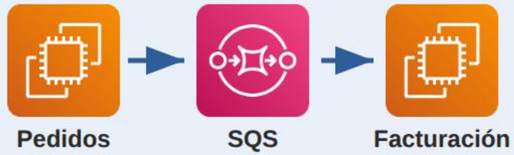
B debe estar disponible justo ahora, o A falla

Tiempo: juntos · **Lugar: conocido**

Frágil ante fallos

en cadena

Cola de mensajes



B puede estar caído un rato

Lo que A sabe de B

A conoce solo el nombre de la cola

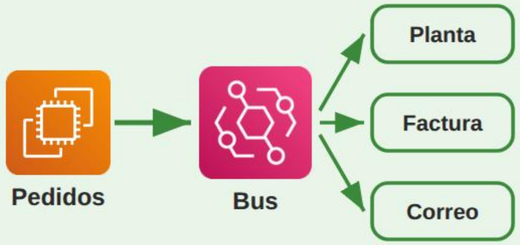
Cada lado avanza a su propio ritmo

Tiempo: libre · **Lugar: libre**

Resiliente y escalable

a costa de complejidad

Evento publicado



A no sabe quién escucha

Lo que A sabe de B

A no conoce a los consumidores

Nuevos suscriptores sin tocar el productor

Tiempo: libre · **Lugar: libre**

Máxima flexibilidad

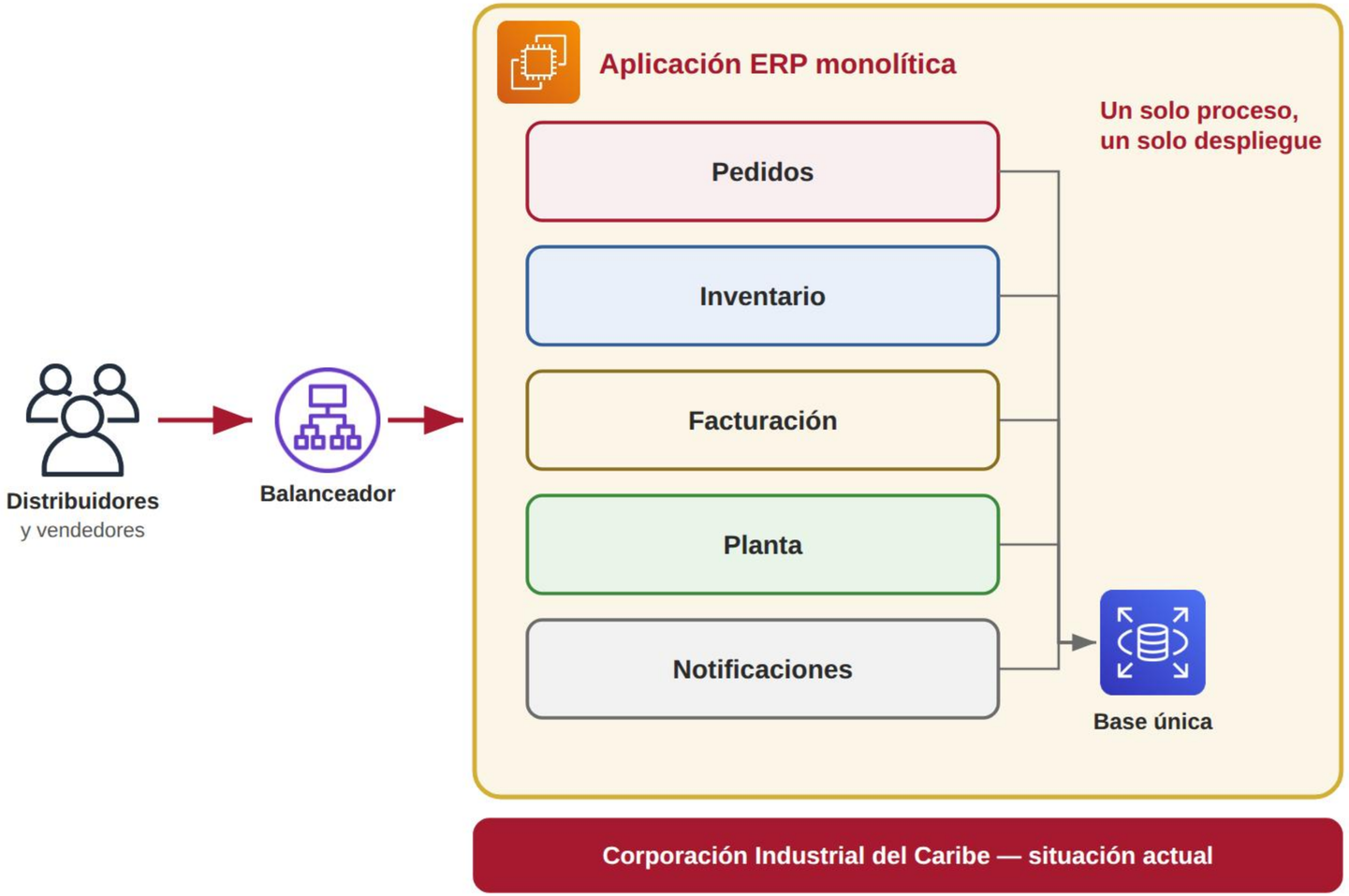
a costa de trazabilidad

Fuente: adaptado de AWS — Implementing Microservices on AWS (docs.aws.amazon.com) · íconos AWS Architecture Icons

CASO

El monolito de la Corporación: todo en una sola pieza

Un solo proceso y una sola base de datos: simple al inicio, rígido cuando el negocio crece.



Corporación Industrial del Caribe — situación actual

1 Despliegue atado

Un cambio pequeño en Facturación obliga a probar y desplegar toda la aplicación

2 Escala atada

Para atender más pedidos hay que clonar también Planta e Inventario

3 Fallos atados

Un error en Notificaciones puede tumbar el proceso completo

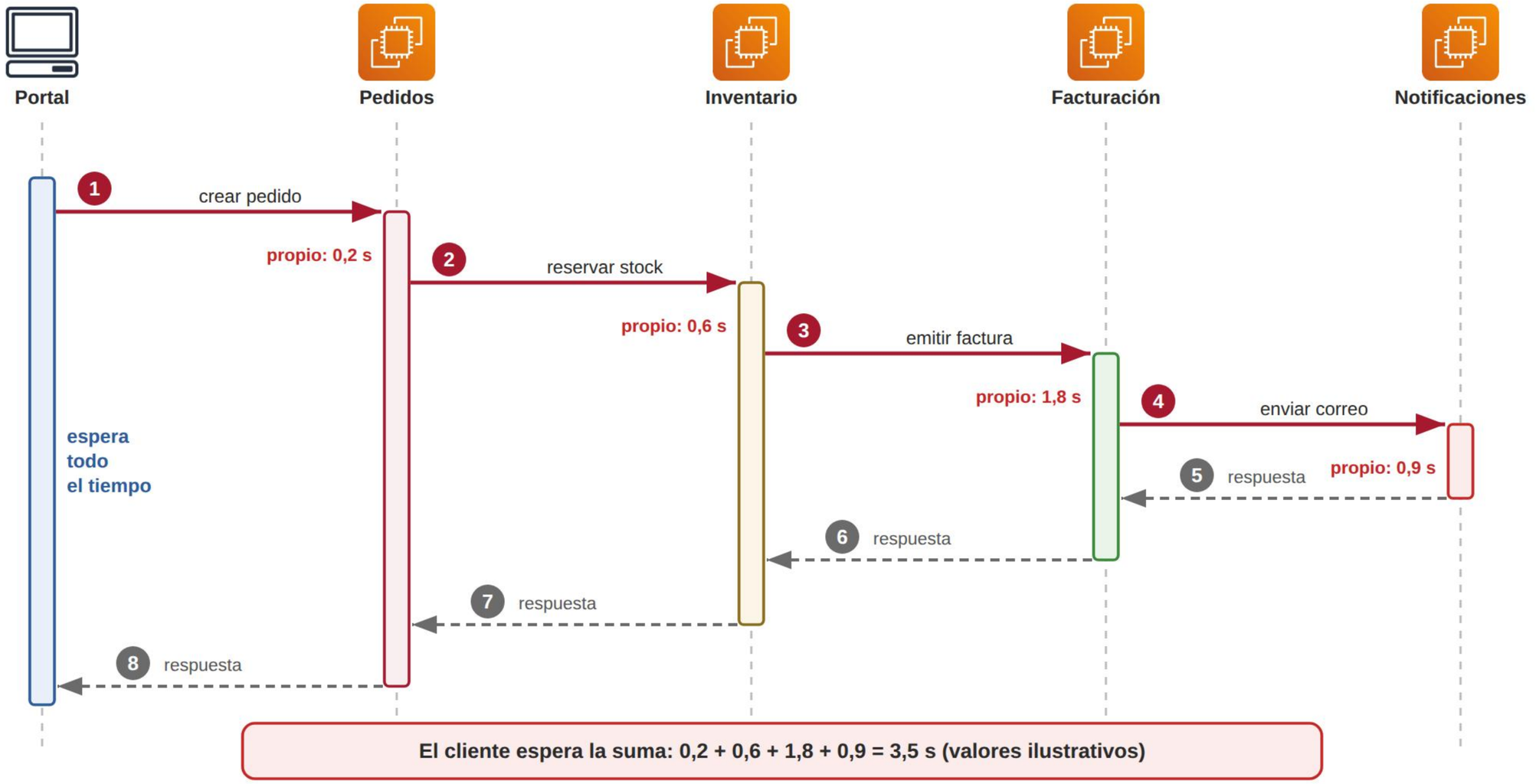
Funciona, pero cada cosa depende de todas las demás

Fuente: adaptado de AWS — Implementing Microservices on AWS (docs.aws.amazon.com) · íconos AWS Architecture Icons

CASO

Llamadas síncronas en cadena: cada paso espera al siguiente

La latencia se suma y el cliente queda atado a la dependencia más lenta de la cadena.

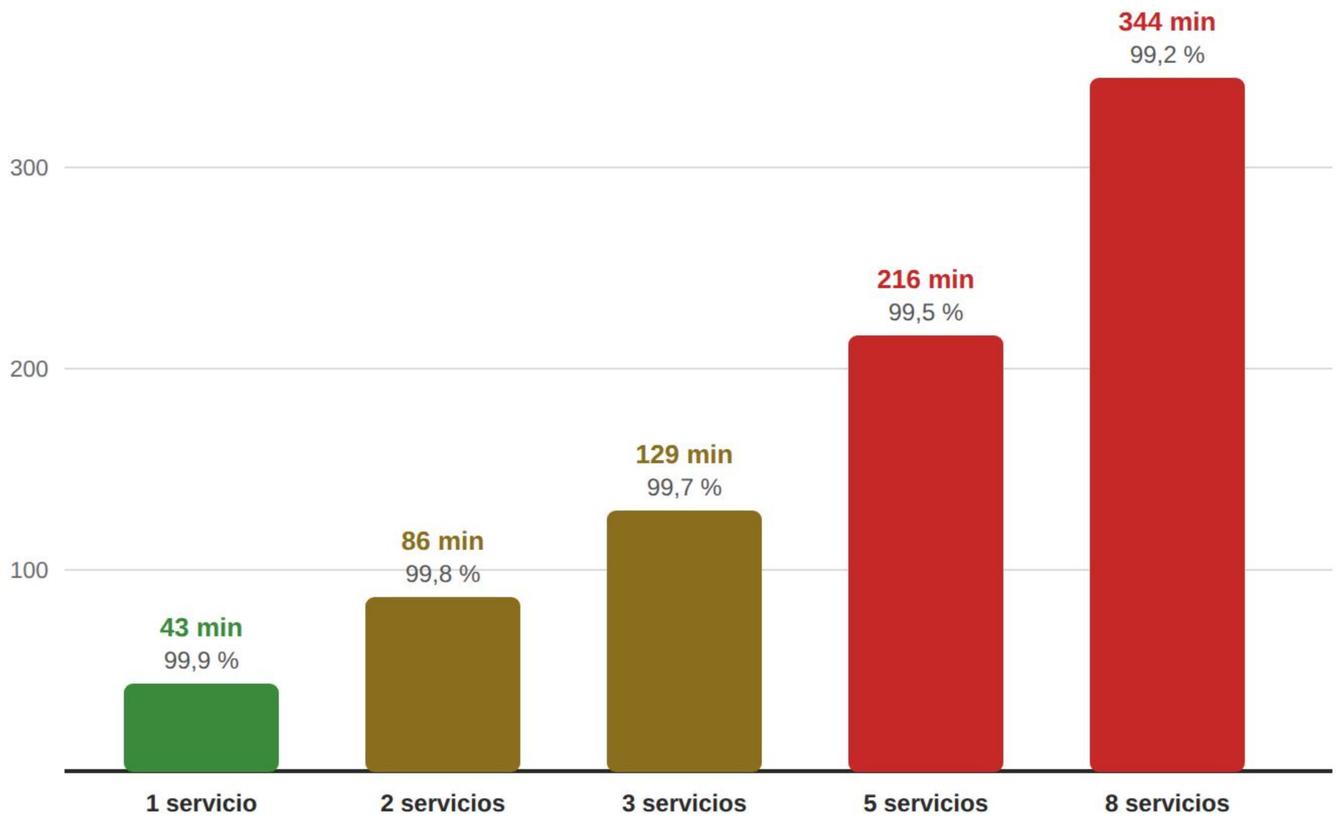


Fuente: adaptado de AWS Well-Architected Framework — Reliability Pillar (docs.aws.amazon.com)

Disponibilidad compuesta: cada eslabón resta

En una cadena síncrona la disponibilidad total es el producto de todas, siempre menor que la peor.

Minutos de caída por mes (30 días)



Cada servicio con 99,9 % de disponibilidad, encadenados en serie

La matemática de la cadena

Disp. total = D1 × D2 × D3 × … × Dn

Ejemplo con cuatro servicios



0,999 × 0,999 × 0,999 × 0,999 = 0,996

Disponibilidad efectiva: 99,6 %

Con 8 eslabones: 99,2 %

casi 6 horas de caída al mes

Ningún eslabón es malo por sí solo:
el problema es multiplicarlos en serie.
Desacoplar rompe la serie.

Fuente: adaptado de AWS Well-Architected — Reliability Pillar, disponibilidad de dependencias duras (docs.aws.amazon.com)

Fallo en cascada: así cae todo por un eslabón lento

La saturación se propaga hacia atrás porque cada servicio mantiene hilos y conexiones esperando.

1 Facturación se vuelve lenta



Base saturada

Hilos ocupados

70 %

tiempo: 0 s

2 Inventario espera y se bloquea



Sus hilos quedan esperando

Hilos ocupados

90 %

tiempo: +15 s

3 Pedidos agota su grupo de hilos



Ya no atiende peticiones nuevas

Hilos ocupados

100 %

tiempo: +30 s

4 El portal recibe timeouts



Los usuarios reintentan

Respuestas 504

45 %

tiempo: +45 s

5 El balanceador declara no sanas las instancias



Todo el sistema queda fuera

Instancias sanas

0 de 2

tiempo: +60 s

Un componente lento es peor que uno caído: quien falla rápido libera recursos; quien tarda los retiene.

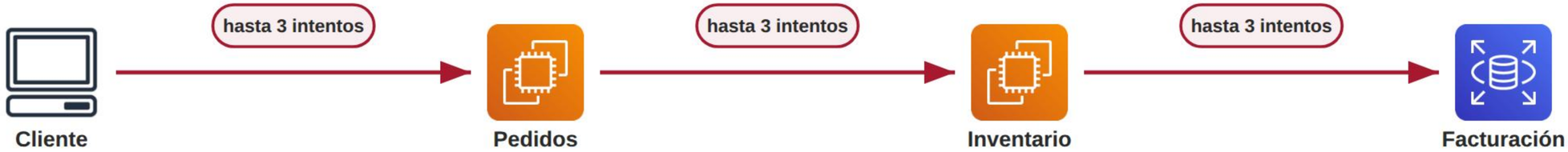
El daño viaja río arriba: de la dependencia más lenta hacia el usuario final.

Tiempos y porcentajes ilustrativos de un incidente típico.

Fuente: adaptado de Amazon Builders Library — Avoiding fallback in distributed systems (aws.amazon.com/builders-library)

Tormenta de reintentos: el cliente empeora el incendio

Si cada capa reintenta, el servicio enfermo recibe la carga multiplicada justo cuando menos puede.



origen: 1 petición

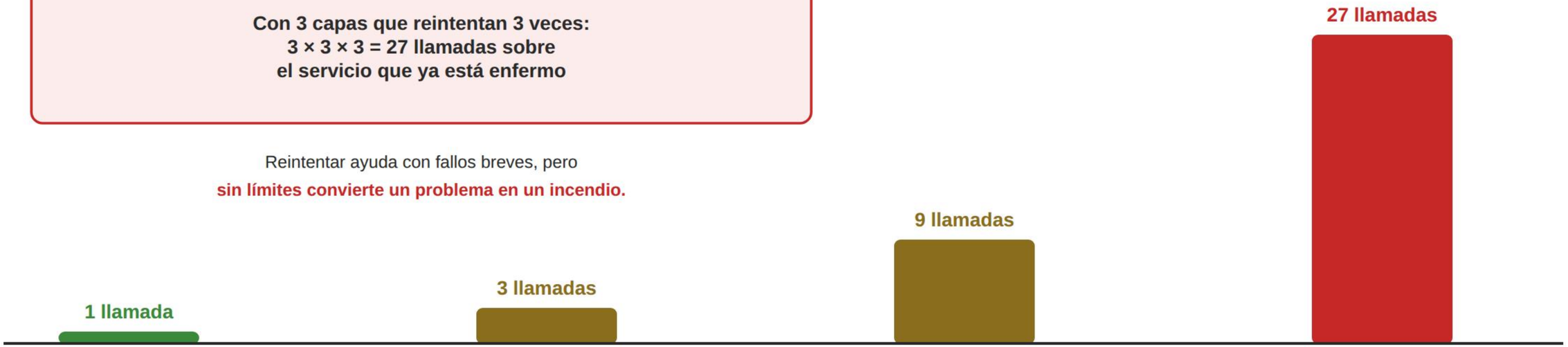
recibe por cada petición original:

recibe:

recibe:

Con 3 capas que reintentan 3 veces:
3 × 3 × 3 = 27 llamadas sobre
el servicio que ya está enfermo

Reintentar ayuda con fallos breves, pero
sin límites convierte un problema en un incendio.



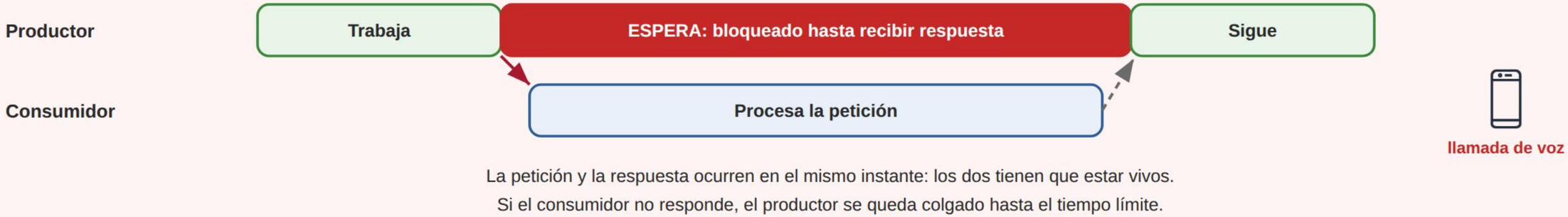
Fuente: adaptado de Amazon Builders Library — Timeouts, retries, and backoff with jitter (aws.amazon.com/builders-library)

CONCEPTO

Comunicación asíncrona: dejar el mensaje y seguir

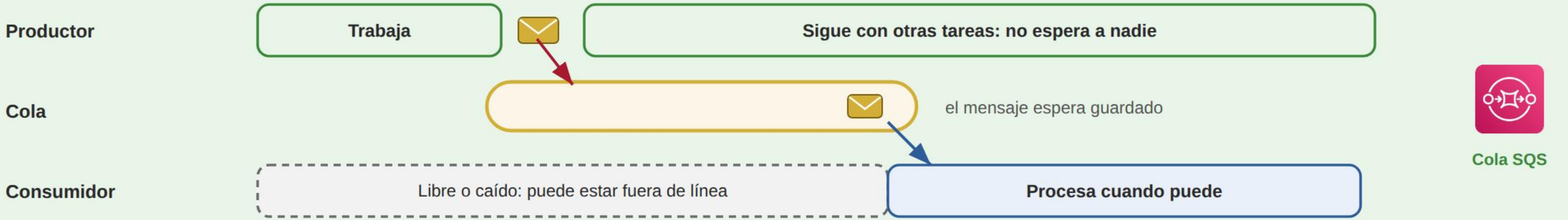
El productor entrega el mensaje a un intermediario y continúa; el consumidor lo toma cuando puede.

Comunicación síncrona: llamar por teléfono y esperar en línea



La petición y la respuesta ocurren en el mismo instante: los dos tienen que estar vivos.
Si el consumidor no responde, el productor se queda colgado hasta el tiempo límite.

Comunicación asíncrona: mandar un audio por WhatsApp y seguir con tu día



Cada lado avanza a su ritmo; el intermediario guarda el mensaje hasta que alguien lo tome.

Tiempo →

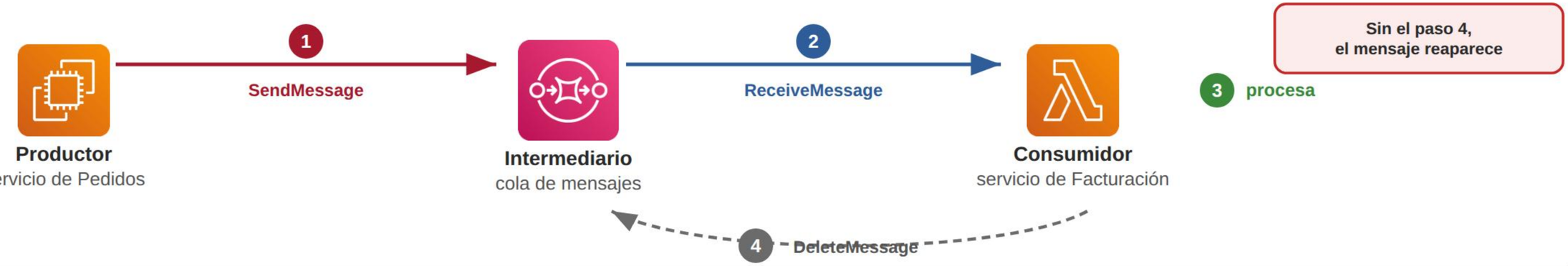
Fuente: adaptado de AWS Prescriptive Guidance — Cloud design patterns, mensajería asíncrona (docs.aws.amazon.com)

CONCEPTO

Productor, mensaje, intermediario y consumidor

Cuatro actores y cuatro pasos: enviar, recibir, procesar y confirmar borrando el mensaje.

Los cuatro actores



Anatomía de un mensaje

Identificador: a1b2-… · lo asigna la cola

CUERPO: lo que el consumidor necesita para trabajar

```
{ "pedido": "CIC-1042", "cliente": "DIST-0731", "items": 12, "total": 18400000 }
```

ATRIBUTOS (metadatos): `tipo = nacional` · `prioridad = alta` · `versión = 1`

Todo en texto (por ejemplo JSON). El total es un valor ilustrativo en pesos.

Qué aporta el intermediario

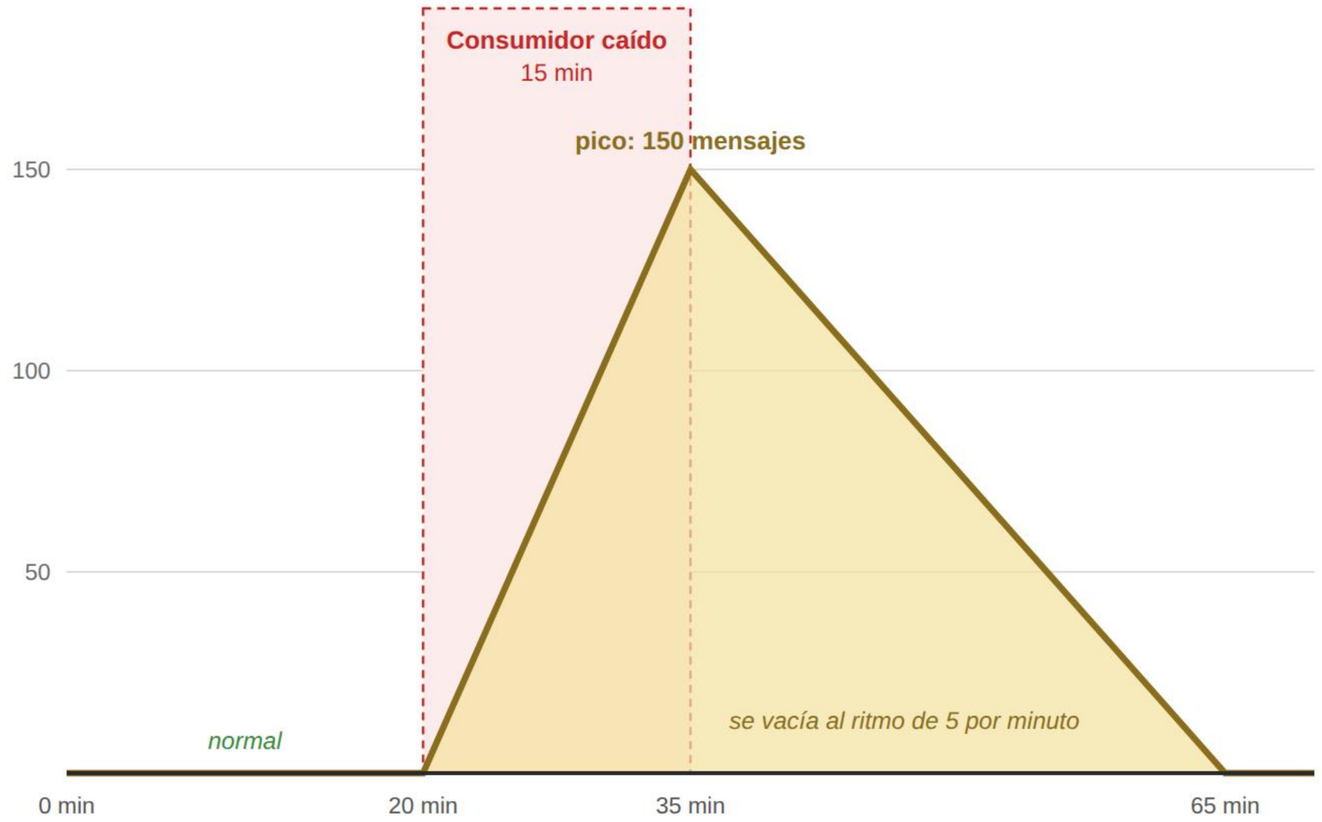
Almacena	guarda el mensaje de forma duradera
Entrega	cada mensaje llega a un consumidor
Aísla	productor y consumidor no se conocen

Fuente: adaptado de Amazon SQS Developer Guide — How queues work (docs.aws.amazon.com)

Beneficio uno: la cola absorbe la caída del consumidor

Mientras el consumidor está caído, los mensajes se acumulan; al volver, se procesan sin pérdida.

Mensajes en la cola (profundidad)



Llegan 10 mensajes por minuto; el consumidor procesa 15 por minuto (valores ilustrativos).

Sin cola (llamada directa)



Cada pedido de esos 15 minutos falla
o se pierde: error 500 al usuario.

Con cola SQS



Ningún pedido se pierde: la cola retiene
los mensajes (hasta 14 días) y se drena al volver.

Fuente: adaptado de Amazon SQS Developer Guide — Message retention y CloudWatch metrics (docs.aws.amazon.com)

Beneficio dos: escalar cada pieza por separado

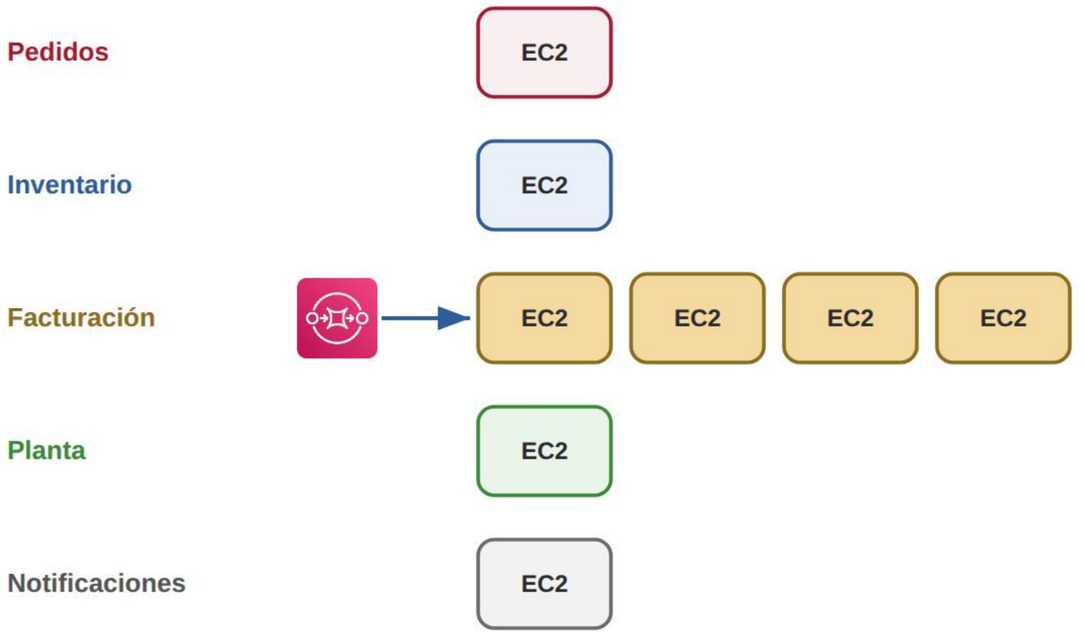
Cada servicio crece según su propia demanda en lugar de clonar la aplicación entera.

Monolito: escalar es clonar todo

Copia 1	Copia 2	Copia 3	Copia 4
Pedidos	Pedidos	Pedidos	Pedidos
Inventario	Inventario	Inventario	Inventario
Facturación	Facturación	Facturación	Facturación
Planta	Planta	Planta	Planta
Notific.	Notific.	Notific.	Notific.

20 unidades de cómputo
para dar 4 veces más capacidad a Facturación

Desacoplado: cada pieza con su tamaño



La cola reparte el trabajo entre las 4 copias de Facturación

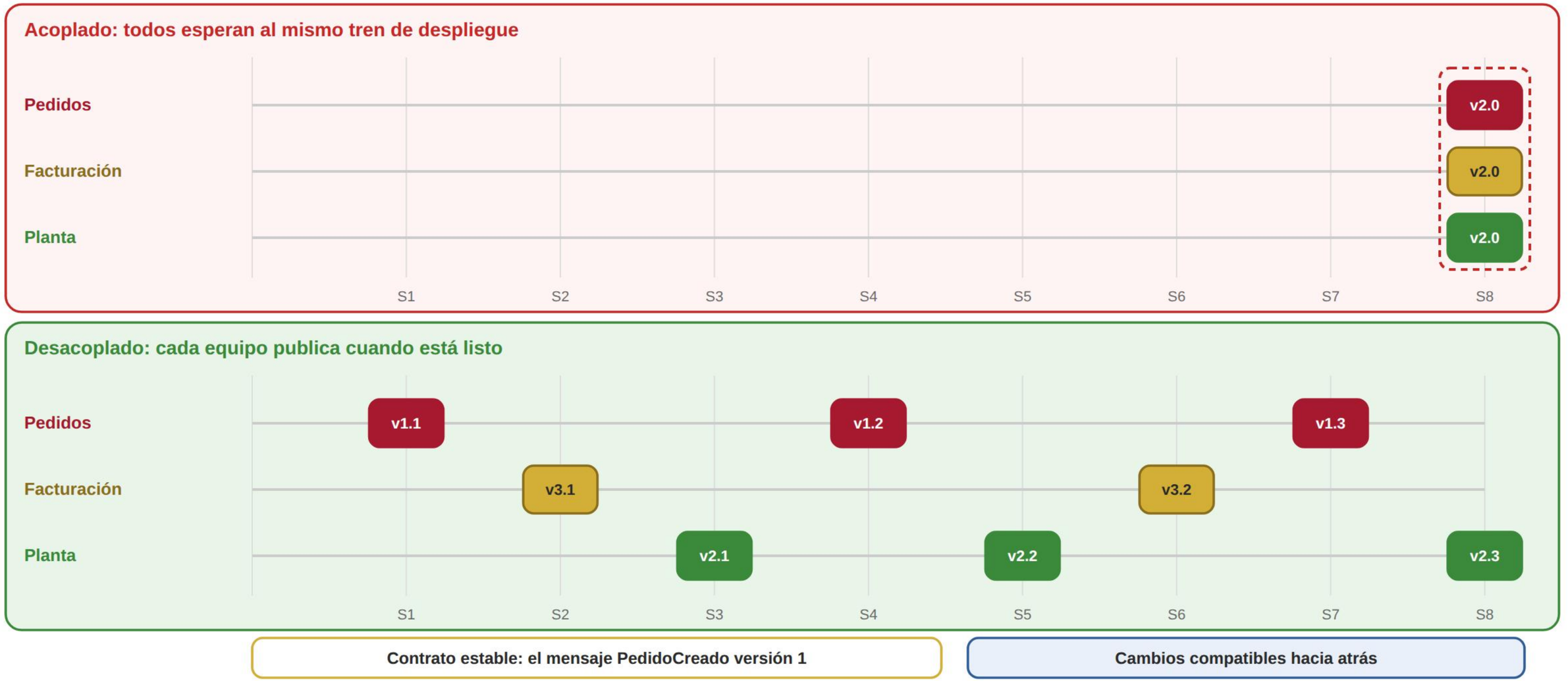
8 unidades de cómputo
escalas solo lo que lo necesita

Cierre de mes en la Corporación: solo Facturación se satura; con el monolito pagas capacidad que no usas.

Fuente: adaptado de AWS Whitepaper — Implementing Microservices on AWS, escalado independiente (docs.aws.amazon.com)

Beneficio tres: equipos que despliegan sin pedir permiso

El mensaje es el contrato: mientras no cambie, cada equipo evoluciona su servicio a su ritmo.

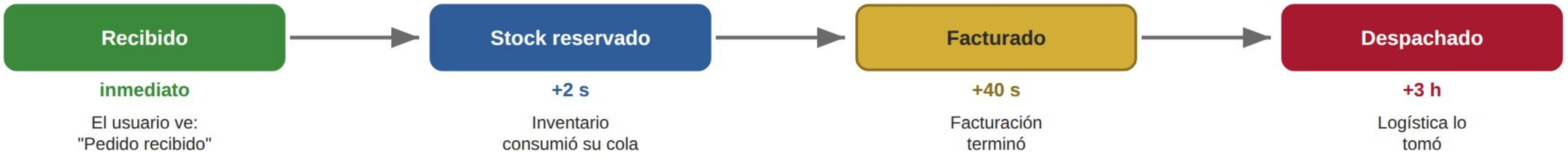


Fuente: adaptado de AWS Whitepaper — Implementing Microservices on AWS, equipos autónomos (docs.aws.amazon.com)

Lo que se paga: consistencia eventual y más piezas

No todo debe ser asíncrono: se elige por la urgencia de la respuesta, y se acepta la complejidad.

Consistencia eventual: el pedido avanza por estados



El sistema converge al estado correcto, pero no todo ocurre en el mismo instante.

Síncrono: necesito la respuesta ahora

- Consultar precio y existencias
- Validar usuario y contraseña
- Calcular el total del carrito

→ quien espera

Asíncrono: puede tardar segundos o minutos

- Emitir la factura electrónica
- Notificar por correo o mensaje
- Generar informes y reportes

→ con cola

Costos nuevos a vigilar: mensajes duplicados, orden no garantizado, y más piezas que monitorizar (idempotencia y trazabilidad: Parte 4).

Fuente: adaptado de AWS Prescriptive Guidance — Saga y mensajería, consistencia eventual (docs.aws.amazon.com)

CIERRE

Lo que queda de la Parte 1

El acoplamiento fuerte convierte un fallo local en un fallo global; el asíncrono lo contiene.

1 Acoplamiento



Cuanto menos sabe A de B, más libres son ambos.

Se mide en dependencias

2 Cadenas síncronas



Latencia sumada, disponibilidad multiplicada y cascadas.

Un eslabón lento tumba todo

3 Asíncrono



Productor, mensaje, cola y consumidor, sin esperar.

El mensaje espera su turno

4 Beneficios



Resiliencia, escala propia y despliegues independientes.

A cambio de complejidad

Siguiente: Parte 2

Colas de mensajes con Amazon SQS

Estándar y FIFO · tiempo de visibilidad · sondeo largo · colas de mensajes fallidos · nivelación de carga





Fuente: síntesis de la Parte 1 — AWS Well-Architected Framework, Reliability Pillar (docs.aws.amazon.com)

SEMANA 13 · ARQUITECTURAS DESACOPLADAS: MENSAJERÍA, COLAS, PUB/SUB

Colas de mensajes con Amazon SQS

Parte 2 de 4 · Unidad 3

Ing. Rodolfo Cañas Cervantes

Universidad de la Costa (CUC) · Ingeniería de Sistemas

Semana 13 · Periodo 2026-2 · Barranquilla, Colombia

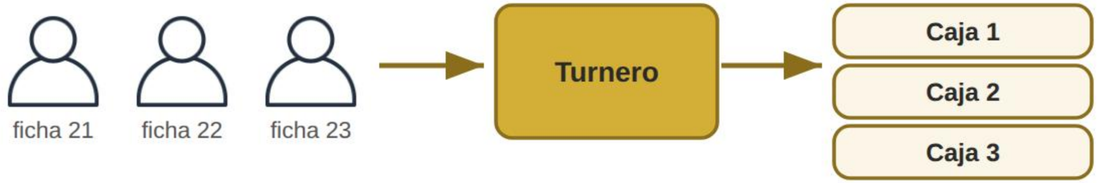
Cómo una cola administrada entrega, protege y reparte el trabajo entre consumidores.

CONCEPTO

Una cola de turnos y una cola de mensajes

La cola desacopla a quien produce trabajo de quien lo atiende, igual que el turnero de un banco.

En el banco: el turnero



Los clientes toman ficha y esperan su turno. Si una caja se cierra, la fila no desaparece: otro cajero atiende la ficha.

ficha	turnero	caja
=	=	=
mensaje	cola	consumidor

En la nube: Amazon SQS



El productor no espera a nadie. Si un consumidor falla, el mensaje sigue en la cola hasta que otro lo atienda.

Servicio administrado: sin servidores que operar, redundante entre zonas y de pago por uso.

Una cola, muchos productores, muchos consumidores

El mismo concepto en otras nubes y en tu centro de datos



Azure Queue Storage
colas simples



Azure Service Bus
colas empresariales



Google Pub/Sub
suscripción pull



OpenStack Zaqar
mensajería de la nube privada



RabbitMQ
broker de código abierto



Amazon MQ
ActiveMQ y RabbitMQ

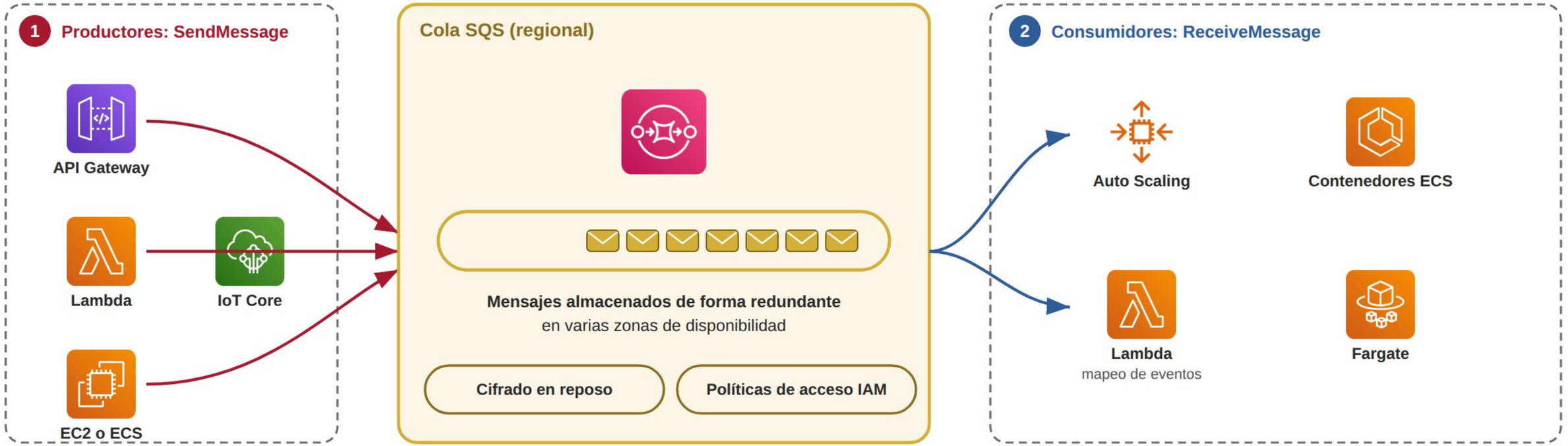
Cambian los nombres y los límites; el patrón de productor, cola y consumidor es el mismo.

Fuente: adaptado de Amazon SQS Developer Guide, Azure Service Bus y Google Pub/Sub (documentación oficial) · íconos de proveedores

SERVICIO

Anatomía de Amazon SQS

Productores envían, la cola guarda de forma redundante y los consumidores toman el trabajo.



4 días
retención por defecto
(de 1 minuto a 14 días)

1 MiB
tamaño máximo
de un mensaje

30 s
tiempo de visibilidad
por defecto

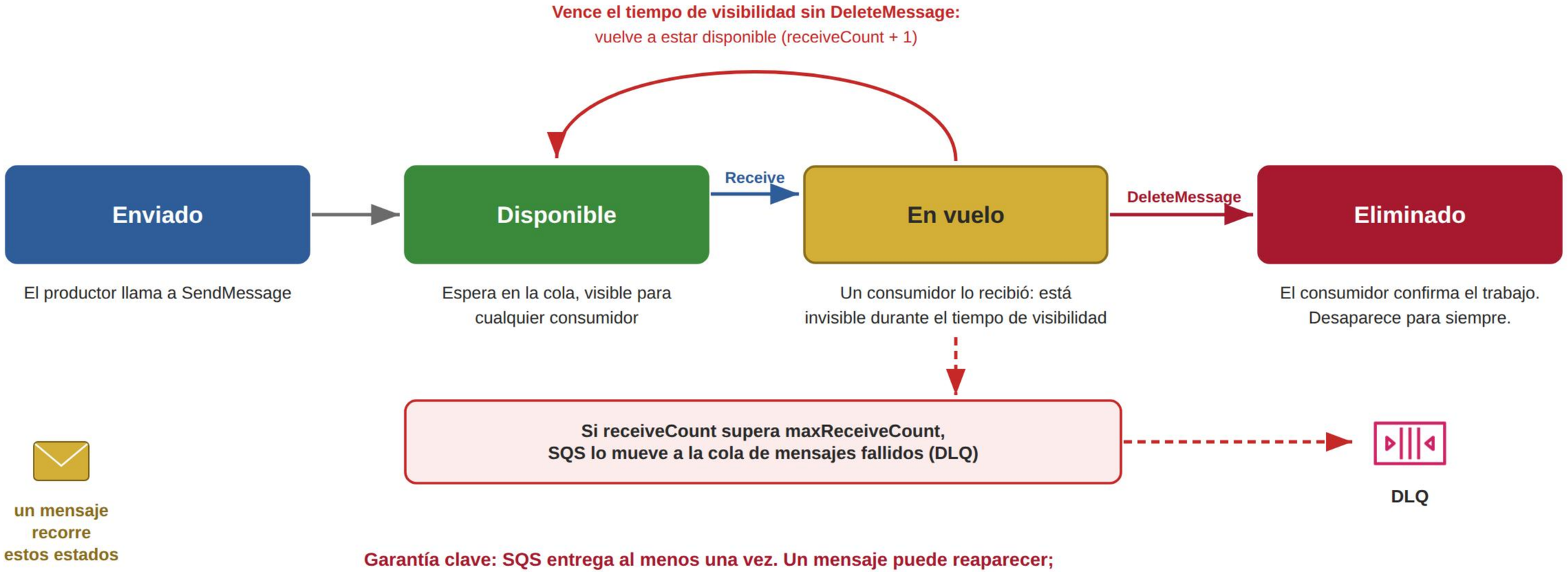
20 s
espera máxima del
sondeo largo

Fuente: adaptado de Amazon SQS Developer Guide (docs.aws.amazon.com) · íconos AWS Architecture Icons

SERVICIO

Ciclo de vida de un mensaje en SQS

Un mensaje no desaparece al leerse: queda invisible hasta que el consumidor lo confirma borrándolo.



Garantía clave: SQS entrega al menos una vez. Un mensaje puede reaparecer; por eso el consumidor debe poder procesarlo dos veces sin daño (idempotencia, Parte 4).

Eliminado, expirado por retención o movido a la DLQ: los tres finales posibles.

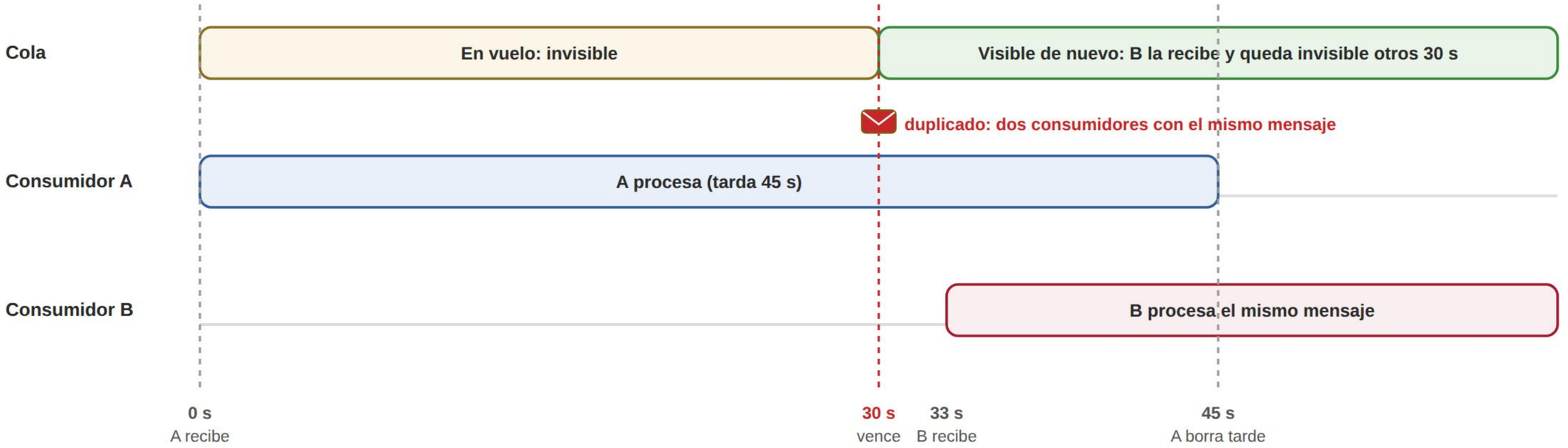
Fuente: adaptado de Amazon SQS Developer Guide — Message lifecycle (docs.aws.amazon.com)

SERVICIO

Tiempo de visibilidad: el candado temporal

Si el consumidor tarda más que el tiempo de visibilidad, otro recibe el mismo mensaje.

Línea de tiempo con tiempo de visibilidad de 30 s, y un procesamiento que tarda 45 s



Cómo evitarlo

- **Visibilidad mayor que el tiempo de procesamiento**
- **ChangeMessageVisibility: extender si se necesita más**
- **Rango: de 0 s a 12 horas (30 s por defecto)**
- **Consumidor idempotente: un duplicado no hace daño**

Fuente: adaptado de Amazon SQS Developer Guide — Visibility timeout (docs.aws.amazon.com)

SERVICIO

Sondeo corto y sondeo largo

El sondeo largo espera hasta 20 segundos: menos llamadas vacías, menor costo y menos latencia.

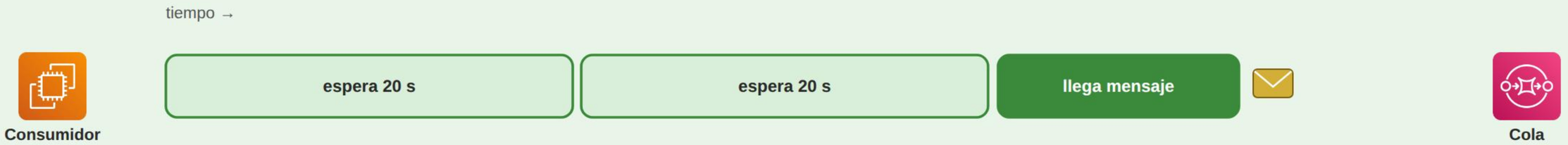
Sondeo corto (short polling): responde de inmediato, aunque esté vacío



24 llamadas en la ventana: 23 respuestas vacías y 1 con mensaje

Más solicitudes, más costo, y puede devolver vacío aunque haya mensajes (consulta un subconjunto de servidores).

Sondeo largo (long polling): espera hasta 20 s a que llegue un mensaje



3 llamadas en el mismo tiempo: menos solicitudes vacías y respuesta casi inmediata

Se activa con WaitTimeSeconds mayor que 0 (hasta 20) o en la propia cola.

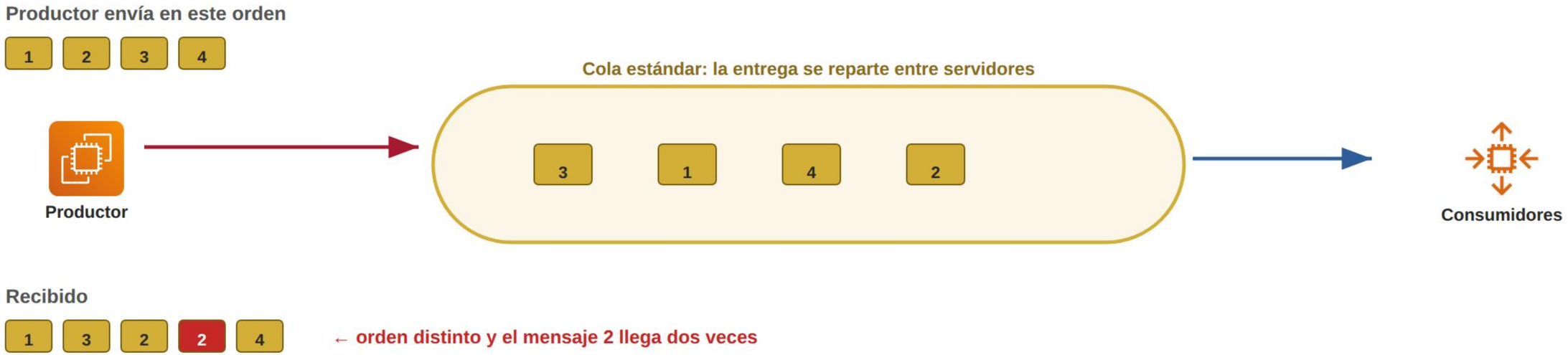
Recomendado casi siempre.

Fuente: adaptado de Amazon SQS Developer Guide — Short and long polling (docs.aws.amazon.com)

SERVICIO

SQS estándar: máximo rendimiento, orden aproximado

La cola estándar escala sin límite práctico, a cambio de duplicados ocasionales y orden no garantizado.



Rendimiento casi ilimitado

Miles de mensajes por segundo sin configurar capacidad

Ideal para volumen alto

Entrega al menos una vez

Puede haber duplicados: el consumidor debe tolerarlos

Hazlo idempotente

Orden de mejor esfuerzo

Suele respetar el orden, pero no lo garantiza

No dependas del orden

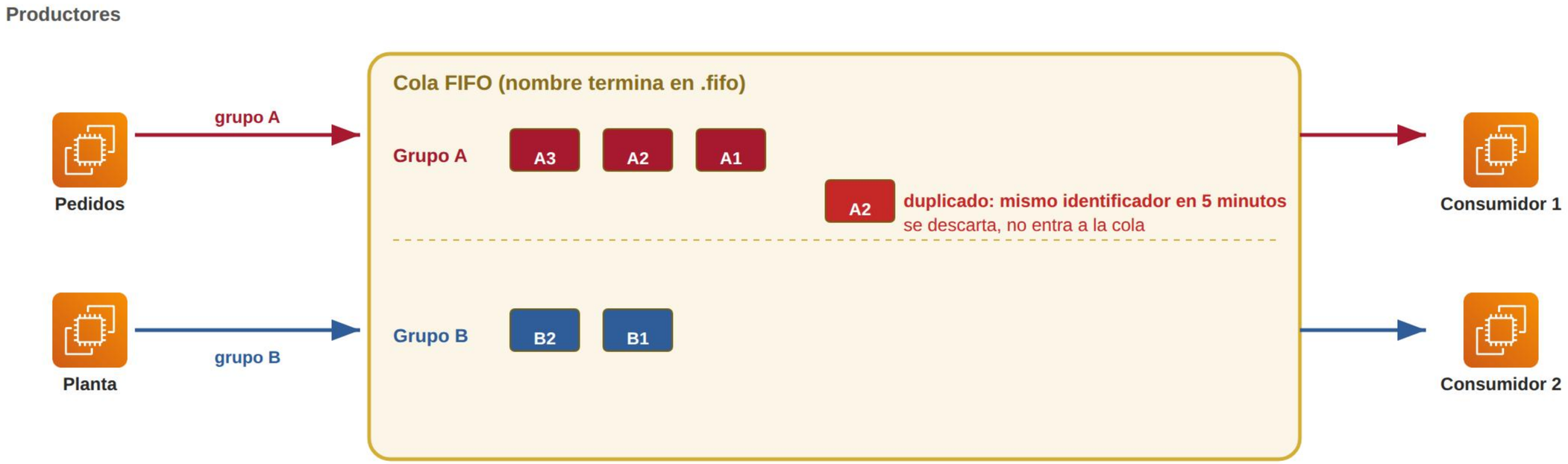
Por defecto elige estándar: notificaciones, procesamiento de imágenes, registros, tareas independientes entre sí.

Fuente: adaptado de Amazon SQS Developer Guide — Standard queues (docs.aws.amazon.com)

SERVICIO

SQS FIFO: orden estricto y sin duplicados

FIFO garantiza el orden dentro de cada grupo de mensajes y descarta duplicados, con menor rendimiento.



MessageGroupId

Define el orden: todo lo del mismo grupo sale en el orden en que entró

Ej.: id del producto

MessageDeduplicationId

Descarta repetidos durante 5 minutos: procesamiento exacto

Ej.: id del pedido

Paralelismo por grupo

Cada grupo lo atiende un consumidor a la vez; varios grupos en paralelo

Más grupos, más paralelismo

Rendimiento menor

300 llamadas por segundo por operación, 3 000 mensajes con lotes

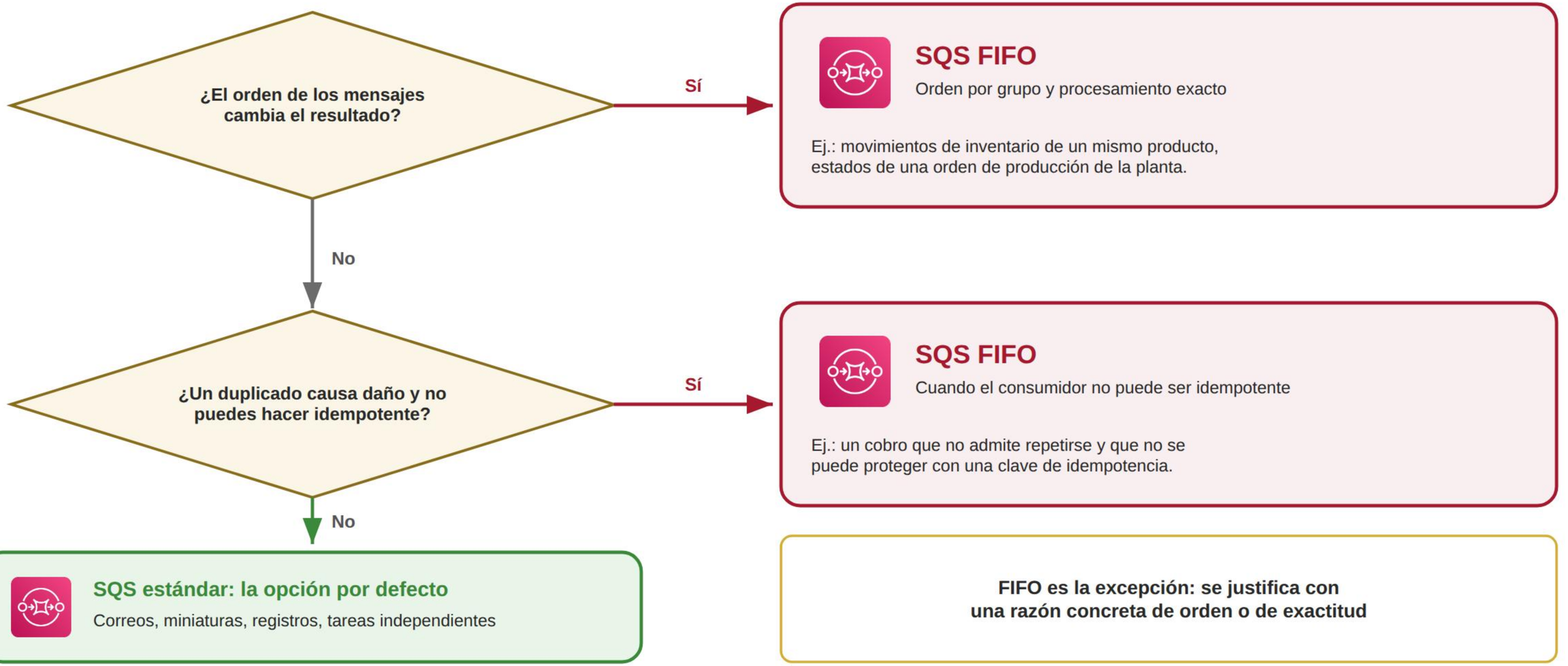
Valores base por defecto

Fuente: adaptado de Amazon SQS Developer Guide — FIFO queues (docs.aws.amazon.com)

DECISIÓN

Estándar o FIFO: cómo elegir

Empieza por estándar; pasa a FIFO solo cuando el orden o la exactitud lo exijan de verdad.



Fuente: adaptado de Amazon SQS Developer Guide — Choosing between standard and FIFO queues (docs.aws.amazon.com)

Colas de mensajes fallidos (DLQ): aparcar al mensaje veneno

Tras superar maxReceiveCount, SQS aparta el mensaje a otra cola para que no bloquee a los demás.



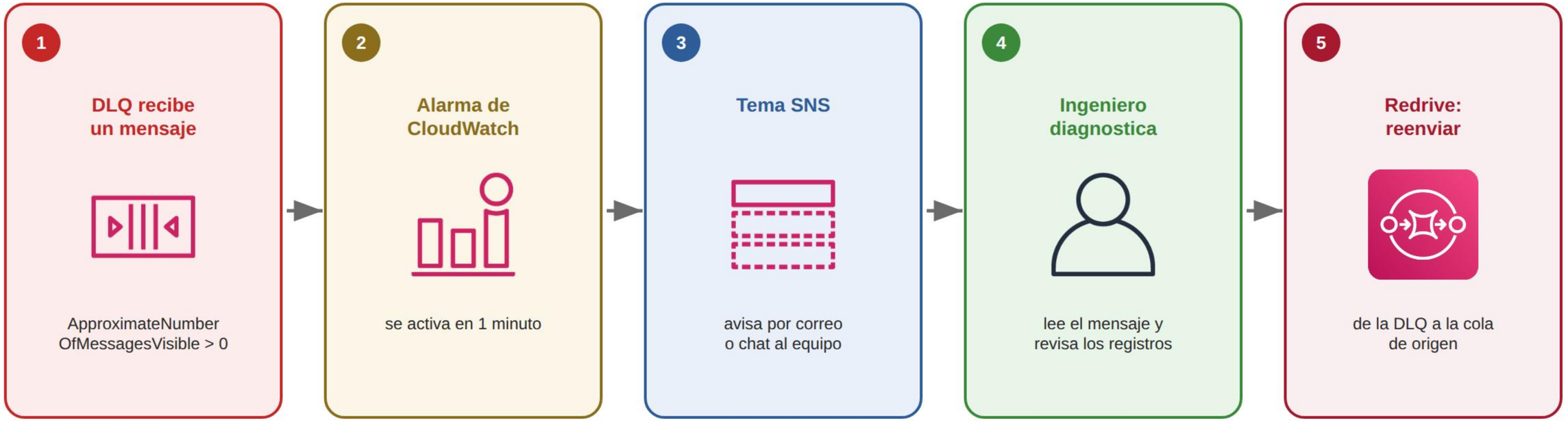
Reglas prácticas

1. **La DLQ es otra cola SQS del mismo tipo**
   estándar con estándar, FIFO con FIFO
2. **Dale retención más larga que la cola de origen:**
   el reloj cuenta desde que el mensaje se envió por primera vez
3. **Un mensaje veneno no bloquea a los demás**
   maxReceiveCount entre 3 y 5 suele bastar

Fuente: adaptado de Amazon SQS Developer Guide — Dead-letter queues (docs.aws.amazon.com)

Operar la DLQ: alarma, diagnóstico y reproceso

Una cola de mensajes fallidos solo sirve si alguien la vigila y sabe reprocesar lo aparcado.



Reproceso seguro: primero corrige el código o los datos; después mueve los mensajes de vuelta (consola o StartMessageMoveTask)

Una DLQ con mensajes es una señal de alerta, no un cementerio:
si nadie la vigila, los pedidos pendientes se pierden en silencio y el negocio no se entera.

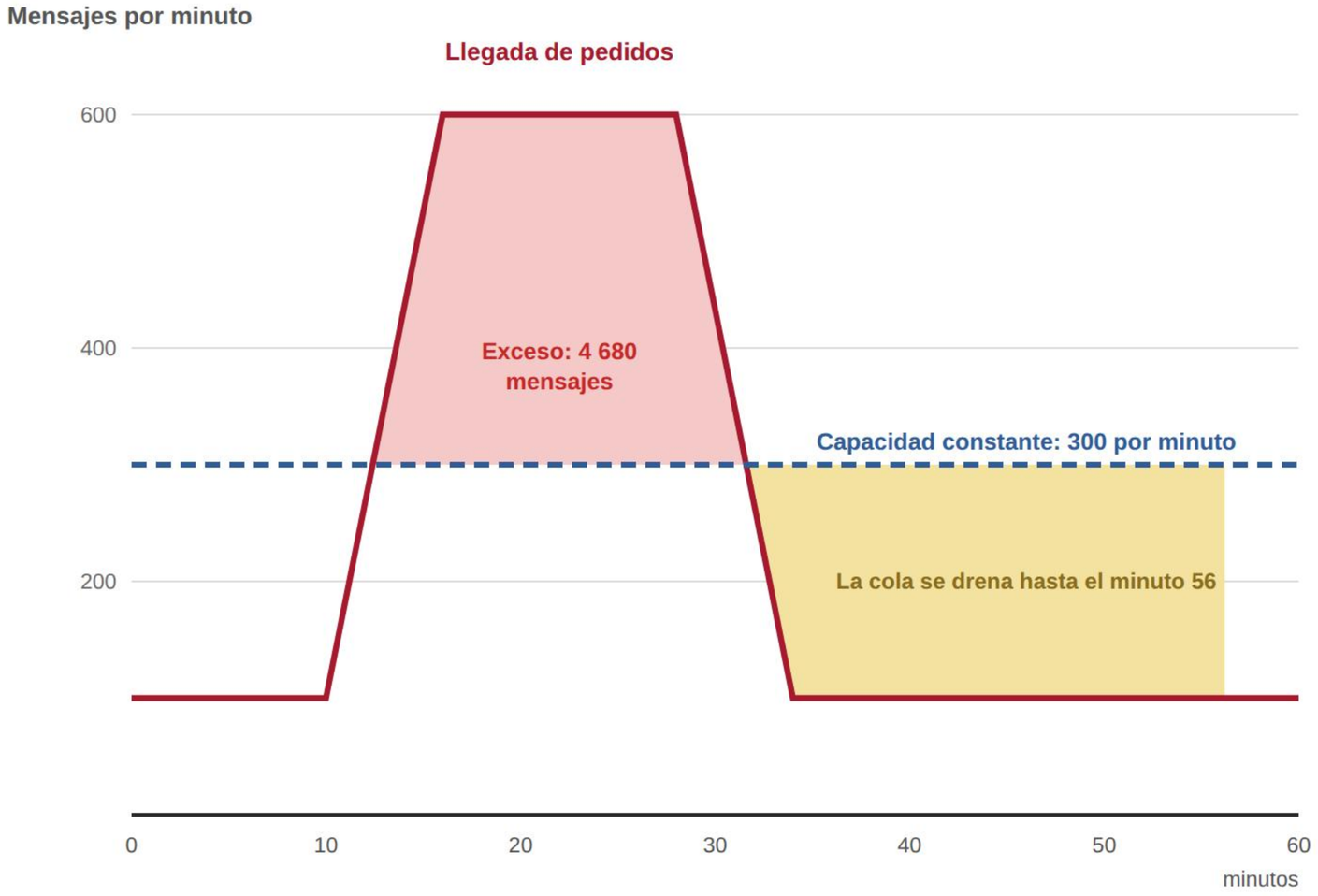
Mide también ApproximateAgeOfOldestMessage en la cola de origen: indica cuánto lleva esperando el mensaje más antiguo.

Fuente: adaptado de Amazon SQS Developer Guide — DLQ redrive y CloudWatch metrics (docs.aws.amazon.com)

PATRÓN

Nivelación de carga: absorber el pico

La cola convierte un pico de llegada en un flujo constante que el consumidor sí puede atender.



Nivelación de carga



El pico no pasa a la base de datos ni al ERP: la cola lo convierte en un flujo estable.

Dimensionas para el promedio, no para el pico

Sin cola: el exceso se rechaza o tumba al consumidor

Con cola: el exceso espera y se procesa cuando hay capacidad

Caso: cierre de mes de la Corporación.

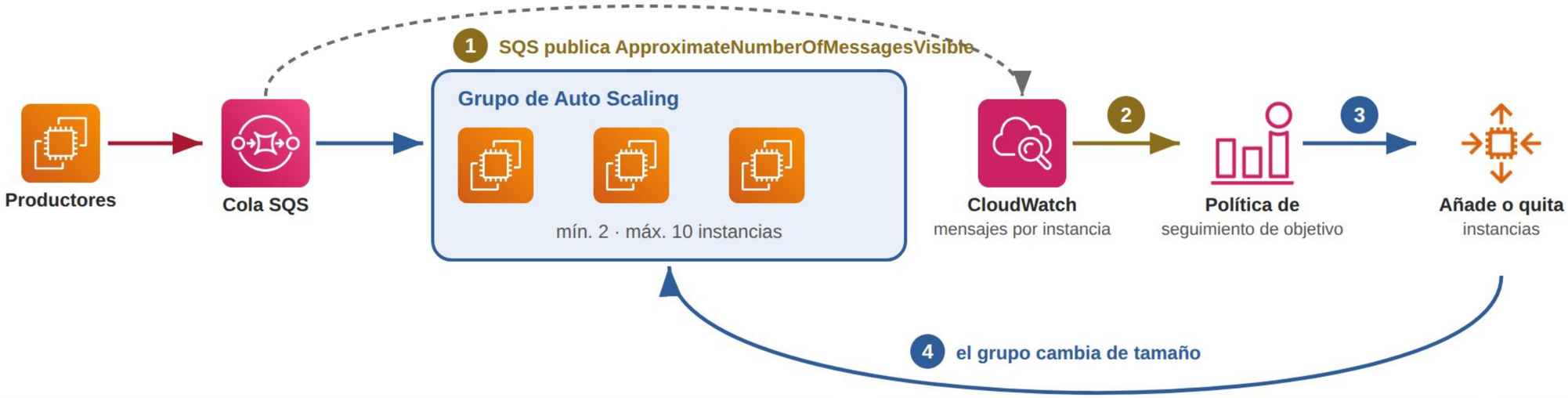
Cifras ilustrativas.

Fuente: adaptado de AWS Well-Architected — Reliability Pillar, patrón de nivelación de carga (docs.aws.amazon.com)

PATRÓN

Escalar consumidores según la profundidad de la cola

La longitud de la cola mide la demanda real; con ella el grupo de instancias crece y se encoge solo.



Cómo fijar el objetivo (ejemplo)

Cada instancia procesa 5 mensajes por segundo

Latencia aceptable: 60 segundos

Objetivo por instancia = 5 × 60 = 300 mensajes

2 400 mensajes ÷ 300 = 8 instancias

La métrica de escala es mensajes en cola por instancia, no la CPU.

Cifras ilustrativas.

Antes

600 mensajes

2 instancias

Cola corta

Pico

2 400 mensajes

8 instancias

Escala hacia fuera

Después

0 mensajes

2 instancias

Escala hacia dentro

Fuente: adaptado de Amazon EC2 Auto Scaling User Guide — Scaling based on Amazon SQS (docs.aws.amazon.com)

CIERRE

Lo que queda de la Parte 2

Con SQS el trabajo espera seguro, se reparte entre consumidores y se vigila con alarmas.

1 Cola SQS



Servicio administrado: enviar, recibir, borrar.

Guarda y entrega

2 Visibilidad y sondeo



Candado temporal y sondeo largo de 20 s.

Menos duplicados y costo

3 Estándar o FIFO



Rendimiento y escala frente a orden y exactitud.

Elige con una pregunta

4 DLQ y nivelación



Aparcar al veneno, absorber picos, escalar por cola.

Resiliencia en acción

Siguiente: Parte 3

Publicación y suscripción, y eventos

SNS · fan-out hacia colas SQS · EventBridge y sus reglas · Azure Service Bus y Event Grid



Fuente: síntesis de la Parte 2 — Amazon SQS Developer Guide (docs.aws.amazon.com)

SEMANA 13 · ARQUITECTURAS DESACOPLADAS: MENSAJERÍA, COLAS, PUB/SUB

Publicación y suscripción, y eventos

Parte 3 de 4 · Unidad 3

Ing. Rodolfo Cañas Cervantes

Universidad de la Costa (CUC) · Ingeniería de Sistemas

Semana 13 · Periodo 2026-2 · Barranquilla, Colombia

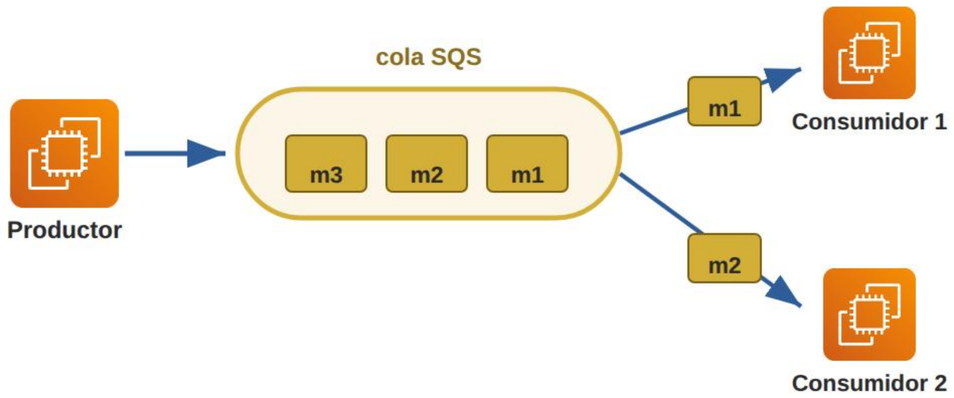
Cómo un solo suceso llega a muchos sistemas sin que el productor los conozca: SNS y EventBridge.

CONCEPTO

De uno a uno, a uno a muchos

La cola reparte tareas entre consumidores; publicación y suscripción difunde una noticia a todos.

Cola: cada mensaje va a UN consumidor



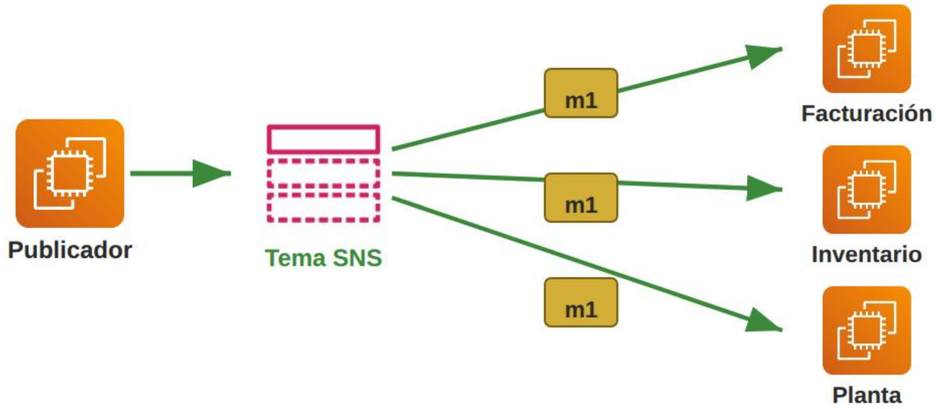
m1 la recibe solo el consumidor 1,
m2 solo el consumidor 2: se reparten el trabajo.

Consumidores que compiten: reparto de trabajo

Úsala cuando el mensaje es una tarea que debe ejecutarse una sola vez.

Ejemplo: emitir una factura.

Publicación y suscripción: a TODOS los suscriptores



m1 la reciben los tres: cada uno
tiene su propia copia del mensaje.

Difusión: un mensaje, muchas copias

Úsala cuando el mensaje es una noticia que interesa a varios sistemas.

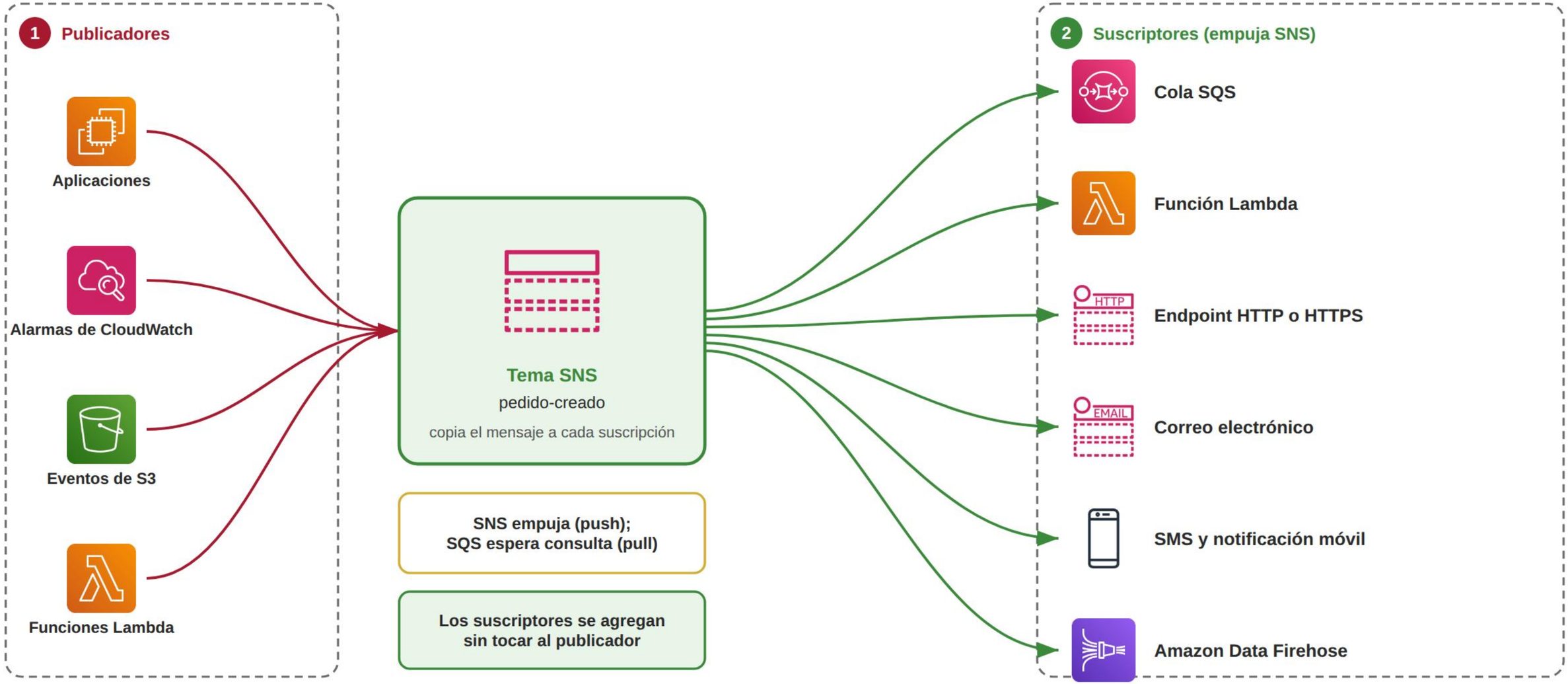
Ejemplo: se creó un pedido.

Fuente: adaptado de AWS Prescriptive Guidance — Publish-subscribe pattern (docs.aws.amazon.com) · íconos AWS Architecture Icons

SERVICIO

Amazon SNS: temas y suscriptores

El publicador envía una vez al tema; SNS entrega una copia a cada suscripción, por empuje.

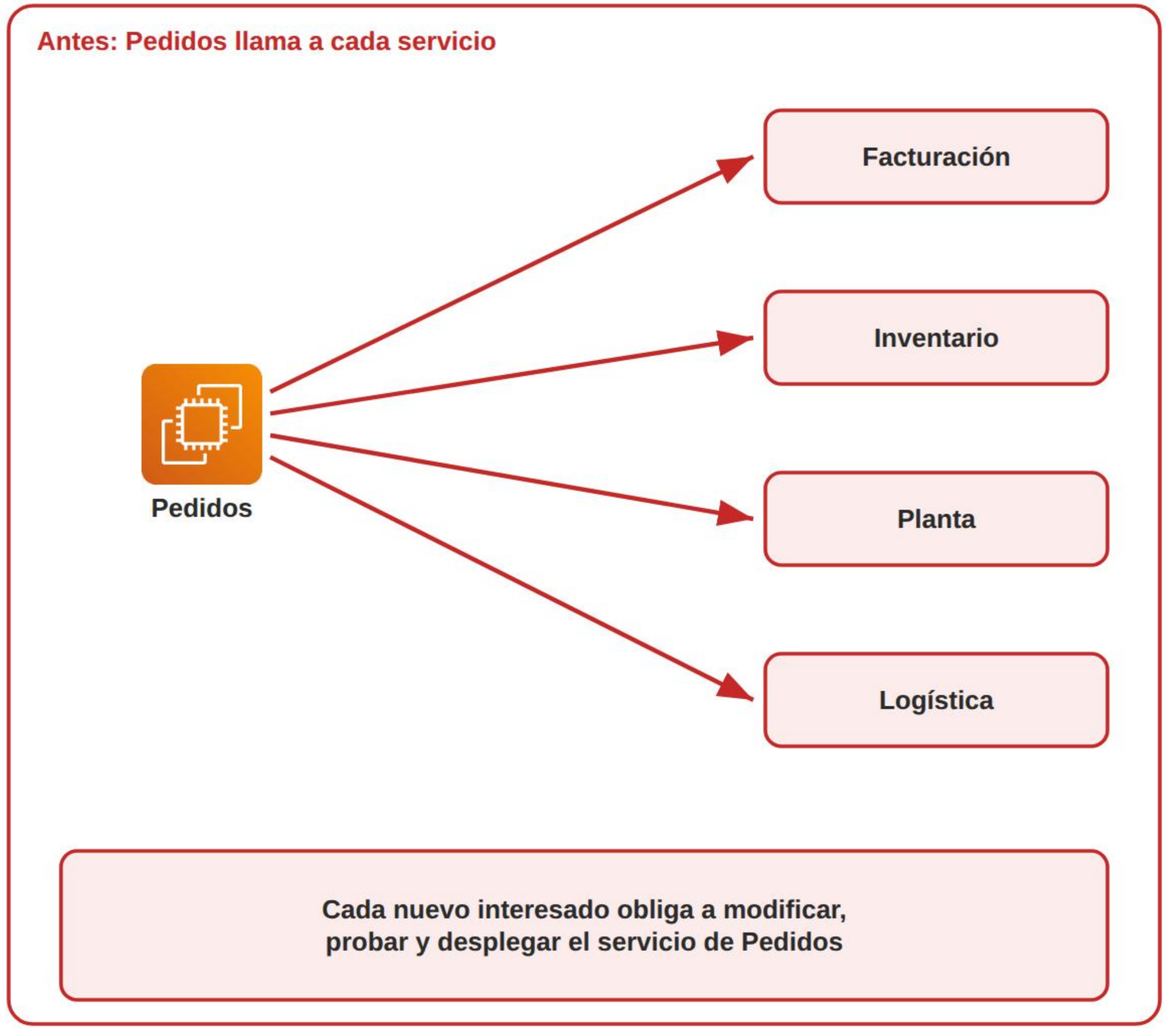


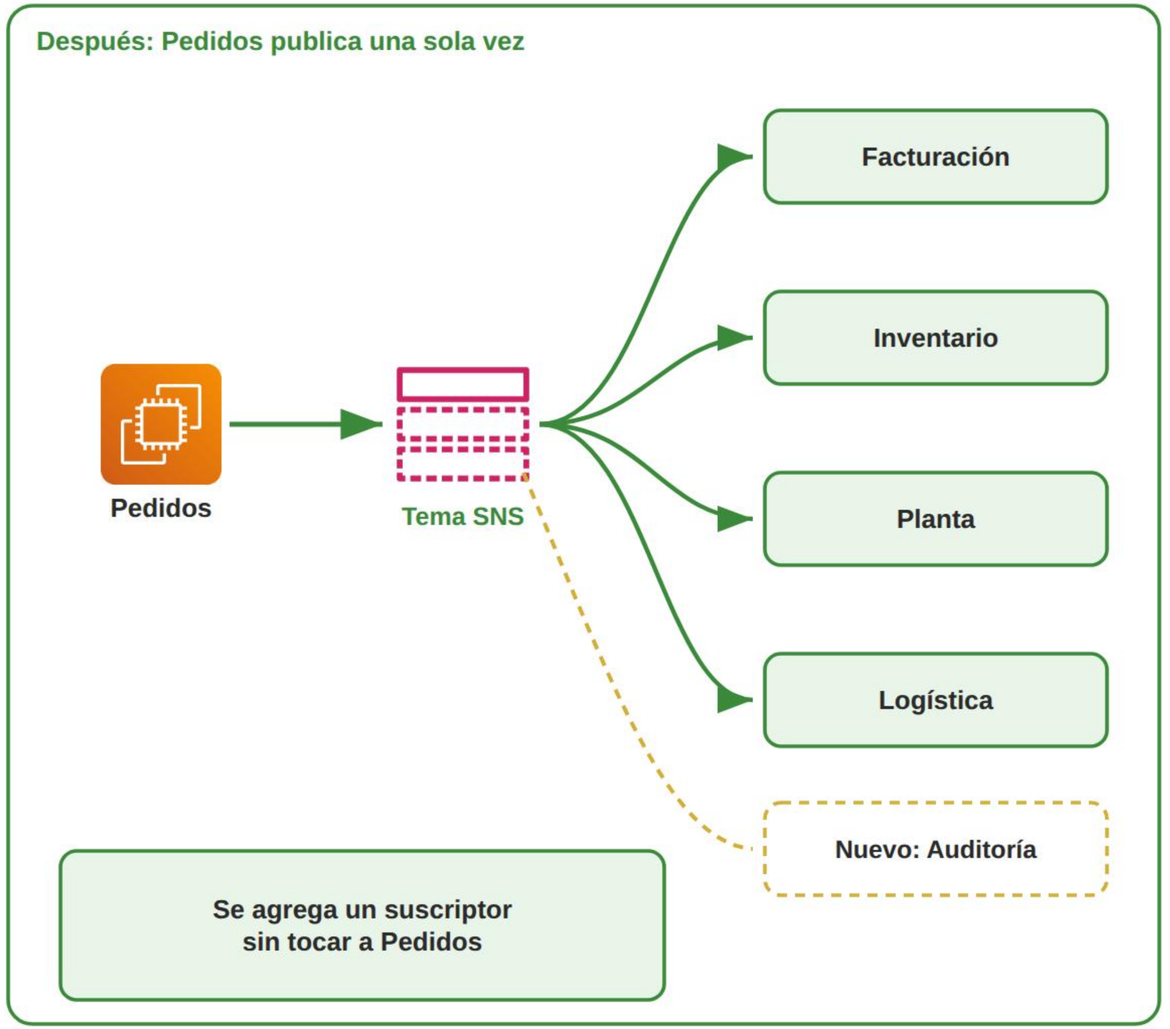
Fuente: adaptado de Amazon SNS Developer Guide (docs.aws.amazon.com) · íconos AWS Architecture Icons

CASO

El problema que resuelve el fan-out

Un solo suceso, varios interesados: publicar una vez evita que Pedidos conozca a todos.



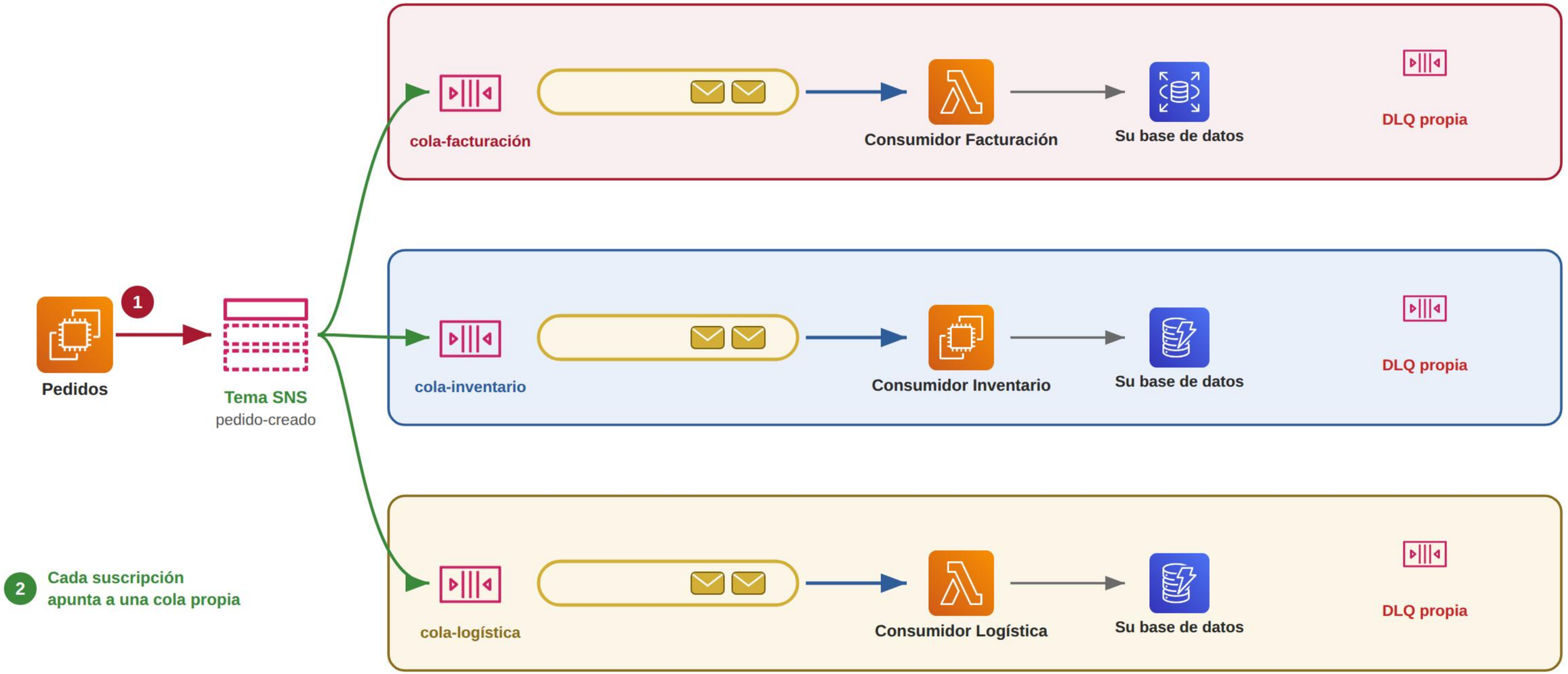


Fuente: adaptado de AWS Prescriptive Guidance — Publish-subscribe pattern (docs.aws.amazon.com) · íconos AWS Architecture Icons

PATRÓN

Patrón fan-out: SNS hacia varias colas SQS

El tema difunde, y cada cola absorbe el trabajo de su consumidor a su propio ritmo.



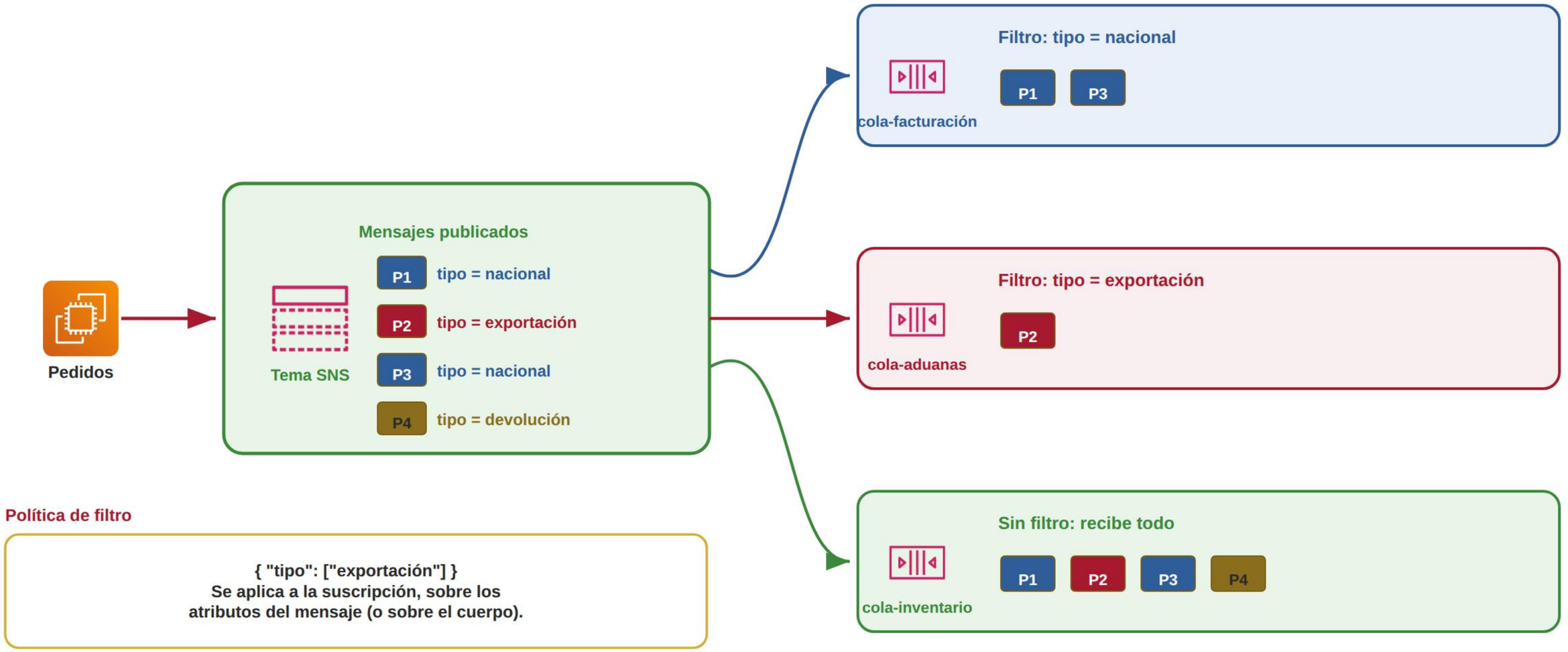
La política de la cola permite que el tema le envíe mensajes. Si un consumidor se cae, su cola retiene el trabajo sin afectar a los demás.

Fuente: adaptado de AWS Architecture Blog y Amazon SNS Developer Guide — Fanout to Amazon SQS queues (docs.aws.amazon.com)

PATRÓN

Filtros de suscripción: cada cola recibe lo suyo

Una política de filtro en la suscripción descarta lo que al suscriptor no le interesa.



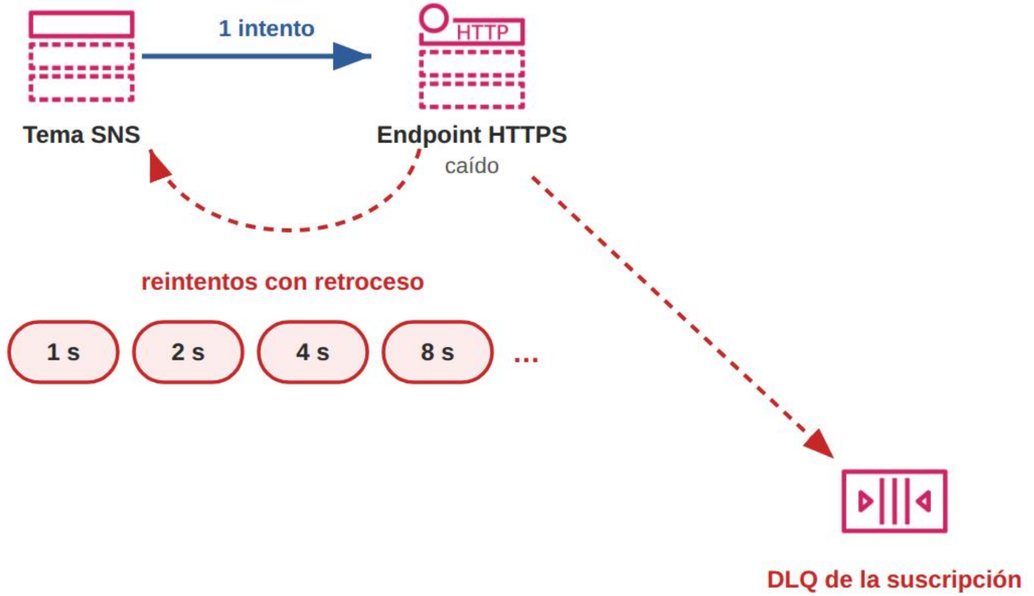
Fuente: adaptado de Amazon SNS Developer Guide — Message filtering (docs.aws.amazon.com) · íconos AWS Architecture Icons

SERVICIO

SNS FIFO y entrega confiable

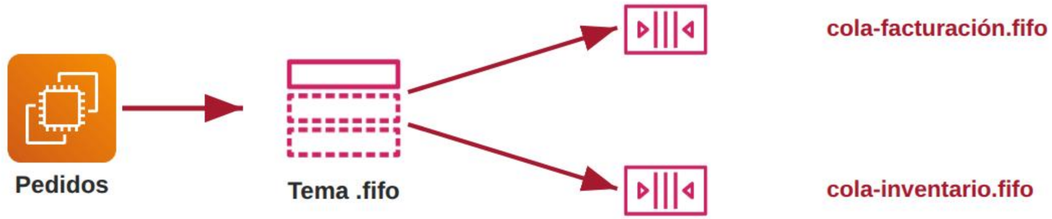
SNS reintenta con retroceso y desvía lo no entregado a una DLQ; el tema FIFO conserva el orden.

Entrega confiable: reintentos y DLQ



Si agota los reintentos de la política de entrega,
SNS guarda el mensaje en una cola SQS (DLQ)
y no se pierde.

SNS FIFO: orden hacia las colas FIFO





El orden del grupo A se mantiene de punta a punta,
y los duplicados se descartan.

Mismo MessageGroupId y deduplicación que en SQS FIFO, ahora desde el tema

Los temas FIFO entregan a colas SQS FIFO: se usan cuando el orden importa para varios consumidores a la vez

Fuente: adaptado de Amazon SNS Developer Guide — Message delivery retries y FIFO topics (docs.aws.amazon.com)

Eventos: algo ocurrió, y a quién le importa

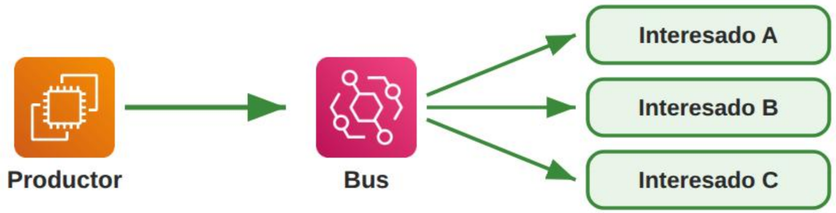
Un evento describe un hecho pasado; quien lo emite no espera respuesta ni conoce a los oyentes.

Comando: una orden dirigida



Verbo en imperativo: espera un resultado

Evento: un hecho ya ocurrido



Verbo en pasado: no espera nada ni sabe quién escucha

Anatomía de un evento en EventBridge (JSON)

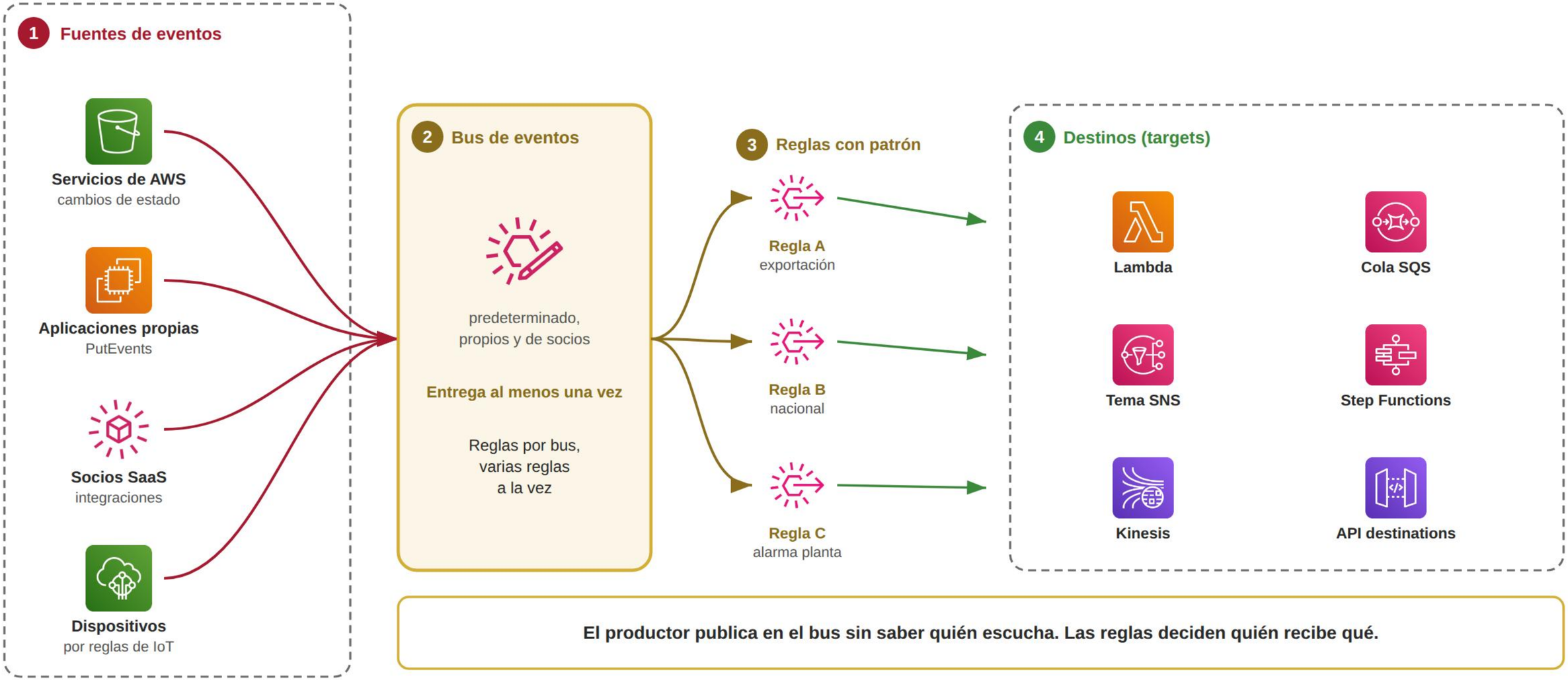
version "0"	*esquema del sobre*
id "6a7e8feb-…"	*identificador único*
detail-type "PedidoCreado"	*qué ocurrió (pasado)*
source "cic.pedidos"	*quién lo emitió*
account · region "123456789012" · "us-east-1"	*cuenta y región*
time "2026-10-12T14:03:07Z"	*cuándo ocurrió*
detail { "pedido": "CIC-1042", "tipo": "exportación", "total": 18400000 }	*los datos del negocio*

Fuente: adaptado de Amazon EventBridge User Guide — Events (docs.aws.amazon.com) · íconos AWS Architecture Icons

SERVICIO

Amazon EventBridge: bus de eventos con reglas

Eventos de AWS, de tus aplicaciones y de socios entran a un bus; las reglas los enrutan a destinos.



El productor publica en el bus sin saber quién escucha. Las reglas deciden quién recibe qué.

Fuente: adaptado de Amazon EventBridge User Guide (docs.aws.amazon.com) · íconos AWS Architecture Icons

SERVICIO

Reglas de EventBridge: patrones que enrutan

La regla compara el evento con su patrón; solo los que coinciden en todo llegan al destino.

Patrón de la regla (JSON)

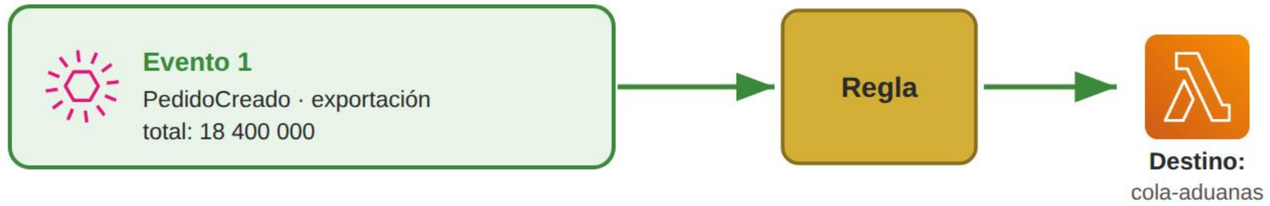
```
{
  "source": ["cic.pedidos"],
  "detail-type": ["PedidoCreado"],
  "detail": {
    "tipo": ["exportación"],
    "total": [{ "numeric": [">", 10000000] }]
  }
}
```

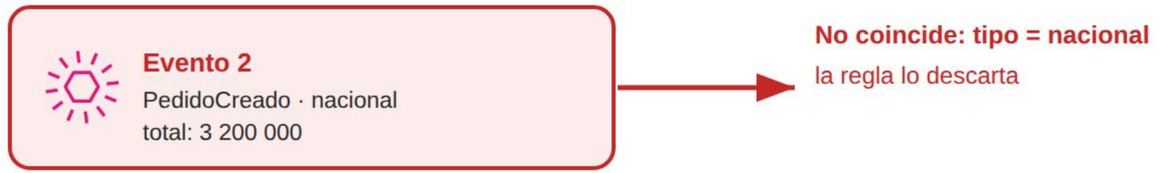
Operadores de contenido

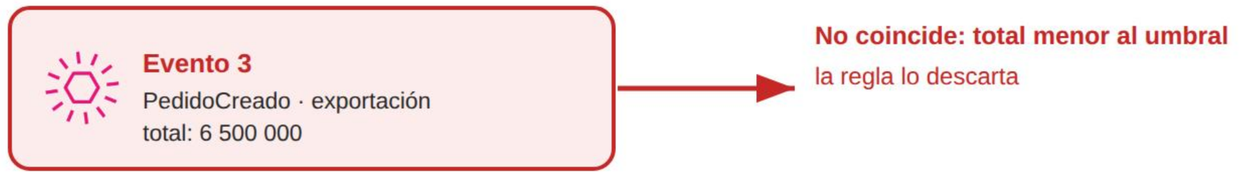
- **igualdad exacta**
- **prefix y suffix**
- **numeric (rangos)**
- **exists**
- **anything-but**
- **or entre valores**

Solo se activa si TODOS los campos del patrón coinciden.
Un campo ausente en el patrón significa "no me importa".

Ejemplo: exportaciones de más de 10 millones de pesos.







Los eventos que no coinciden no llegan al destino:
el consumidor solo recibe lo que declaró que le interesa.

Fuente: adaptado de Amazon EventBridge User Guide — Event patterns (docs.aws.amazon.com) · íconos AWS Architecture Icons

SERVICIO

Archivo, esquemas, Pipes y Scheduler

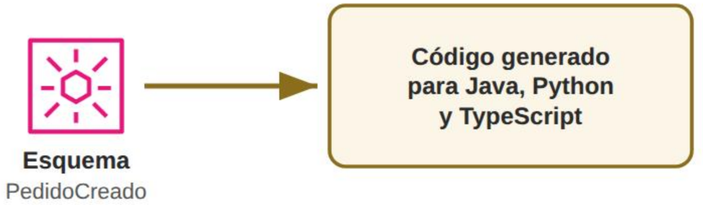
EventBridge no solo enruta: guarda, documenta, conecta y programa eventos.

Archivo y reproducción (archive y replay)



Reproduces eventos pasados tras corregir un error o probar un consumidor nuevo con datos reales

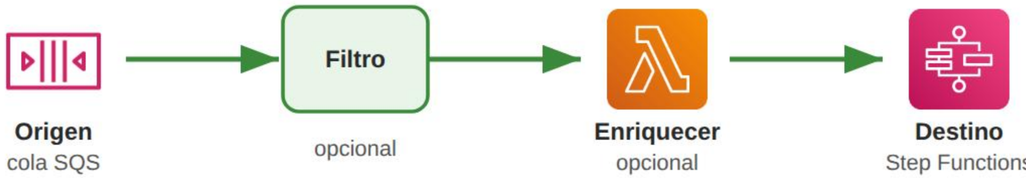
Registro de esquemas (schema registry)



Descubre los eventos y documenta su forma

El contrato del evento queda escrito y versionado

EventBridge Pipes: conectar de punto a punto



Sin código de pegamento: lee de un origen, transforma y entrega a un destino

EventBridge Scheduler: tareas programadas



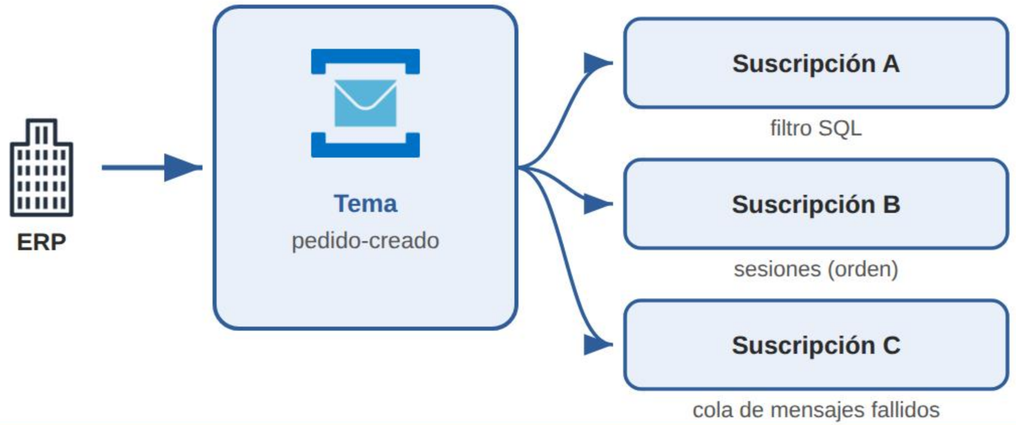
Cierre de mes cada día 30 a las 6 p. m.: un evento programado dispara el cierre contable

Fuente: adaptado de Amazon EventBridge User Guide — Archive, Schema registry, Pipes y Scheduler

El mismo patrón en Azure: Service Bus y Event Grid

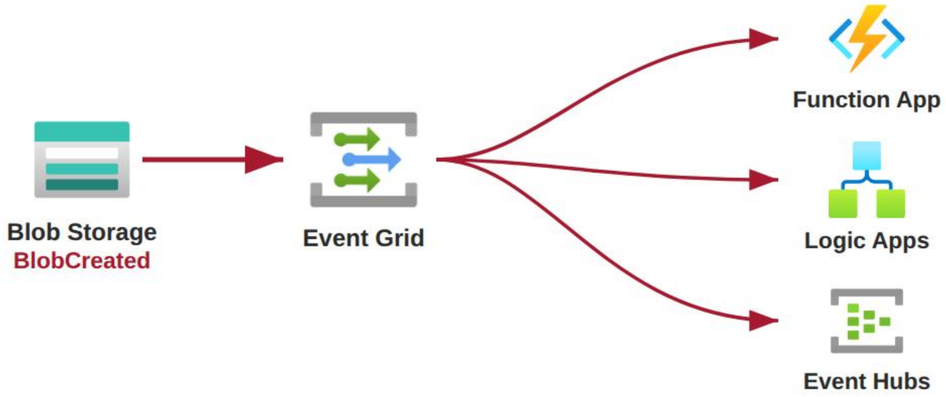
Azure separa mensajes empresariales (Service Bus) de eventos reactivos (Event Grid).

Azure Service Bus: mensajería empresarial



Colas y temas con suscripciones, filtros, sesiones para orden FIFO y cola de mensajes fallidos integrada

Azure Event Grid: eventos reactivos



Notifica hechos (se subió un archivo, se creó un recurso) y los entrega por HTTP a los suscriptores

Y en Google Cloud

Pub/Sub: temas y suscripciones pull o push, con retención y reproducción.

Mismo modelo, otro nombre.

Fuente: adaptado de Azure Architecture Center — Azure messaging services (learn.microsoft.com)

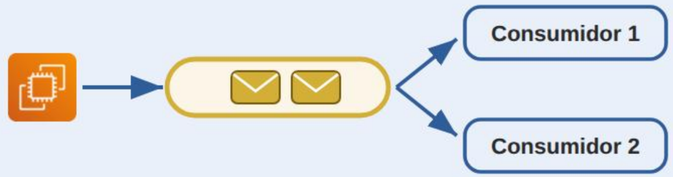
DECISIÓN

Colas, pub/sub y eventos: cuándo usar cada uno

No compiten: se combinan. Lo habitual es un tema o bus que difunde hacia colas que absorben.

Cola (SQS)

Reparte tareas

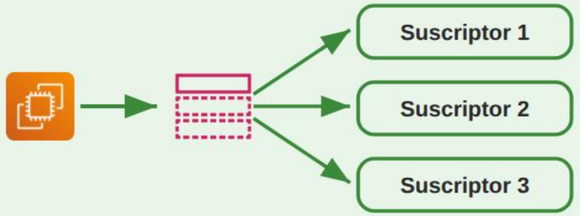


Úsalo cuando…

- Un mensaje es una tarea para un solo consumidor
- Necesitas absorber picos y escalar consumidores
- Quieres reintentos y DLQ por consumidor

Publicación y suscripción (SNS)

Difunde mensajes

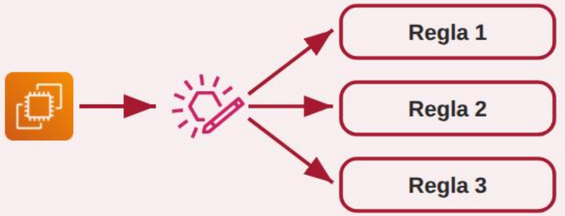


Úsalo cuando…

- Un mensaje interesa a varios sistemas conocidos
- Quieres difundir a colas, Lambda, correo o SMS
- Filtras por atributos y reduces ruido

Bus de eventos (EventBridge)

Enruta por contenido



Úsalo cuando…

- Muchas fuentes: servicios de AWS, SaaS y tus aplicaciones
- Enrutas por el contenido del evento y quieres reproducir
- Programas tareas y conectas con Pipes

Fuente: adaptado de AWS Prescriptive Guidance y Azure Architecture Center — mensajería

CIERRE

Lo que queda de la Parte 3

Una cola reparte, un tema difunde y un bus enruta: el diseño real las combina.

1 Pub/sub



Un mensaje, una copia para cada suscriptor.

Difunde noticias

2 Fan-out



SNS hacia varias colas SQS con filtros y DLQ.

Lo mejor de ambos

3 EventBridge



Bus, reglas con patrón, archivo y replay.

Enruta por contenido

4 Elegir



Cola reparte, tema difunde, bus enruta eventos.

Se combinan

Siguiente: Parte 4

Patrones de robustez y caso integrador

Idempotencia · reintentos con retroceso · orden y duplicados · streaming · Corporación Industrial del Caribe



Fuente: síntesis de la Parte 3 — Amazon SNS y Amazon EventBridge (docs.aws.amazon.com)

SEMANA 13 · ARQUITECTURAS DESACOPLADAS: MENSAJERÍA, COLAS, PUB/SUB

Patrones de robustez y caso integrador

Parte 4 de 4 · Unidad 3

Ing. Rodolfo Cañas Cervantes

Universidad de la Costa (CUC) · Ingeniería de Sistemas

Semana 13 · Periodo 2026-2 · Barranquilla, Colombia

Idempotencia, reintentos con retroceso, orden, streaming y el diseño final de la Corporación Industrial del Caribe.

Entrega al menos una vez: por qué llegan duplicados

Si falla la confirmación, el mensaje reaparece y el trabajo se hace dos veces.



La falla ocurrió entre hacer el trabajo y confirmar. SQS no puede saber si el trabajo se hizo: ante la duda, reentrega.

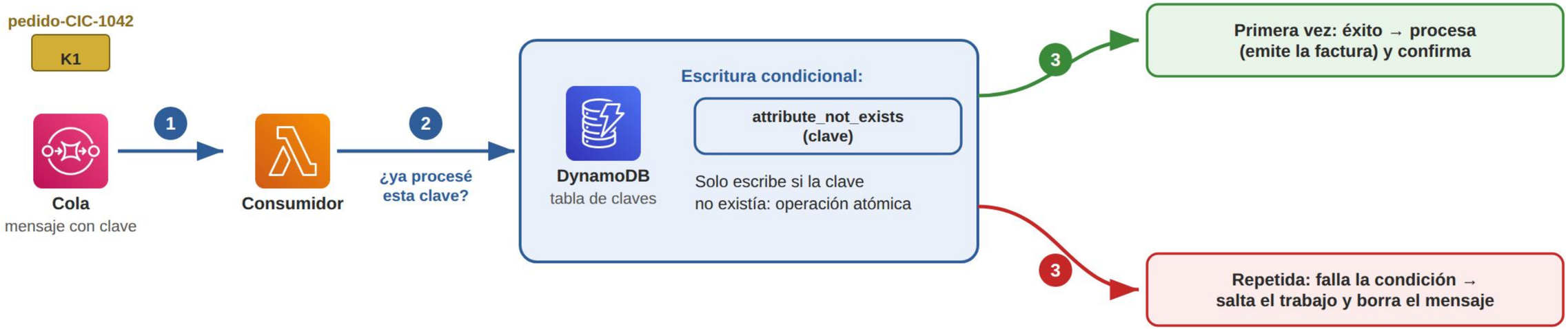
Entrega al menos una vez: la regla en sistemas distribuidos

Es mejor entregar de más que perder un mensaje. Quien consume debe poder recibir el mismo mensaje dos veces **sin cobrar dos veces ni emitir dos facturas. Esa propiedad se llama idempotencia.**

Fuente: adaptado de Amazon SQS Developer Guide — At-least-once delivery y Amazon Builders Library (docs.aws.amazon.com)

Idempotencia: procesar dos veces equivale a procesar una

Una clave de idempotencia y una escritura condicional convierten dos entregas en un solo efecto.



Resultado: dos entregas, un solo efecto

Entrega 1	Entrega 2	Facturas emitidas
factura emitida	reconocida y omitida	para el pedido: 1

Detalles de diseño que cuentan:

la clave sale del negocio (el número de pedido), no del mensaje; y la tabla guarda un tiempo de expiración (TTL) para no crecer sin fin.

Si el consumidor se cae a mitad del trabajo, el registro se marca "en curso" y se reintenta; "terminado" solo al final.

Fuente: adaptado de AWS Prescriptive Guidance y Amazon Builders Library — Making retries safe with idempotent APIs

Operaciones idempotentes y no idempotentes

Diseña las operaciones para asignar un estado, no para acumular un cambio.

No idempotente: aplicar dos veces cambia el resultado

saldo = saldo + 100



Idempotente: aplicar dos veces da lo mismo

estado = "PAGADO"



El duplicado no cambia nada

Tres formas prácticas de conseguirlo

Clave natural

Usa el número de pedido o factura como clave única.

Escritura condicional

Inserta solo si no existe; actualiza solo si el estado coincide.

Deduplicación de FIFO

MessageDeduplicationId descarta repetidos, pero solo 5 minutos.

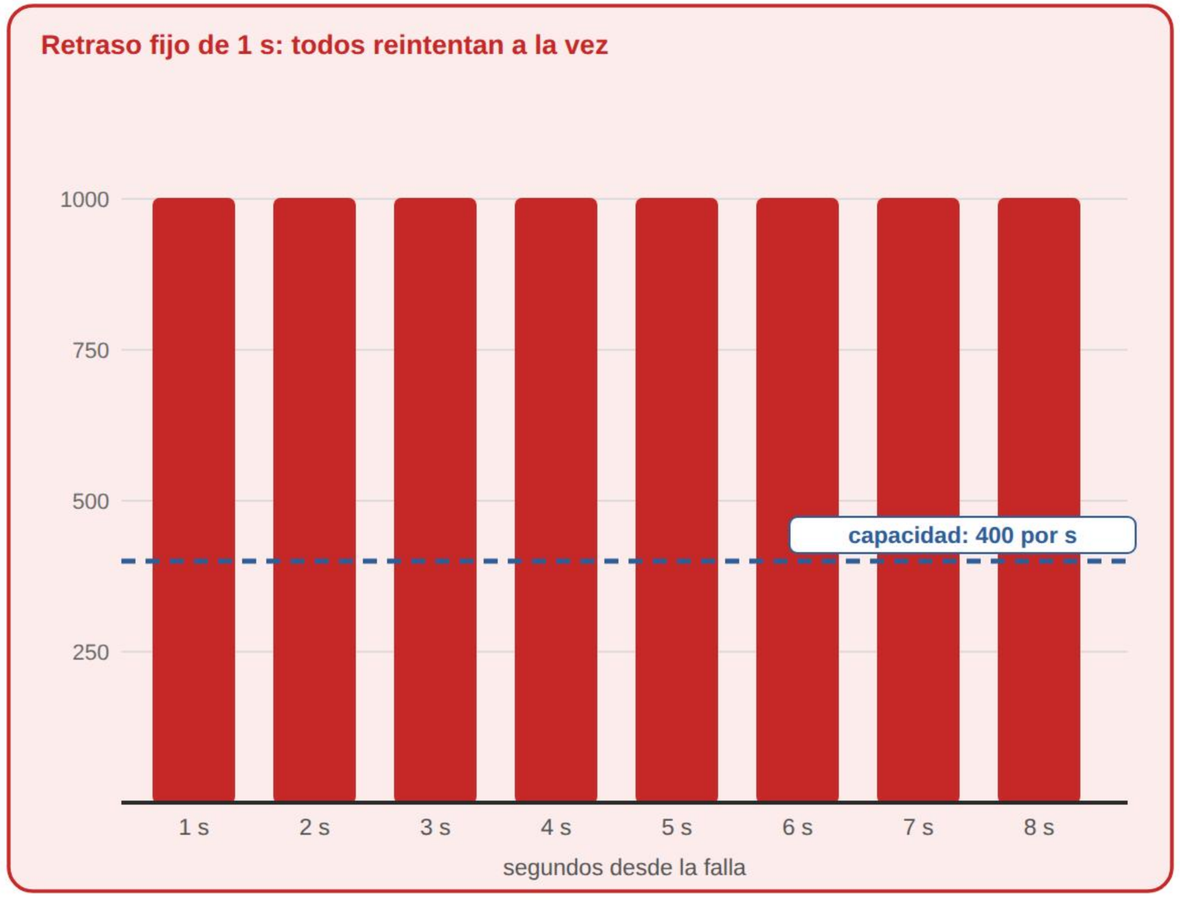
Asignar, no acumular

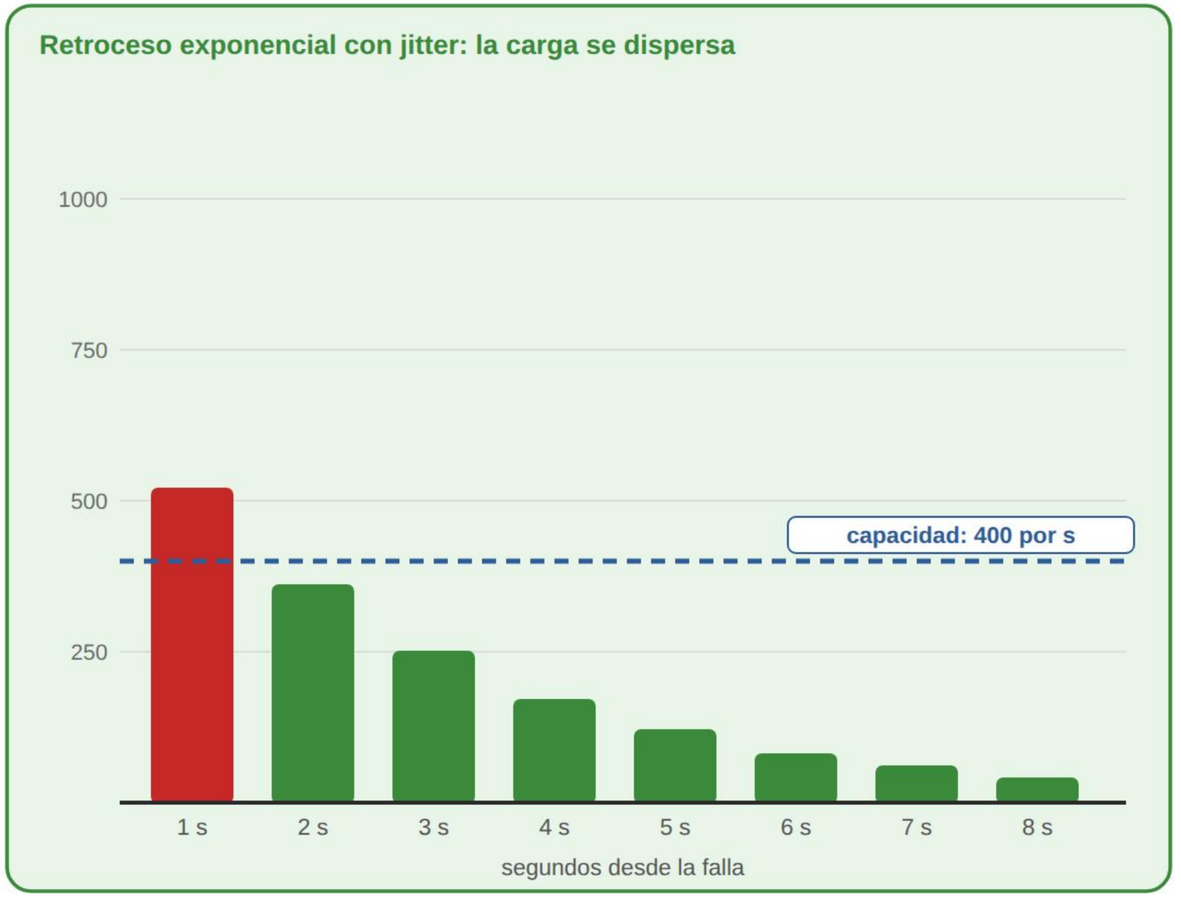
Prefiere "estado = X" y "upsert" a "contador + 1" o "insertar".

Fuente: adaptado de Amazon Builders Library — Making retries safe with idempotent APIs (aws.amazon.com/builders-library)

Reintentos: el retraso fijo sincroniza la avalancha

Reintentar a intervalos fijos mantiene a todos los clientes marchando al mismo paso.





Cada oleada supera la capacidad: el servicio nunca se recupera (efecto manada)

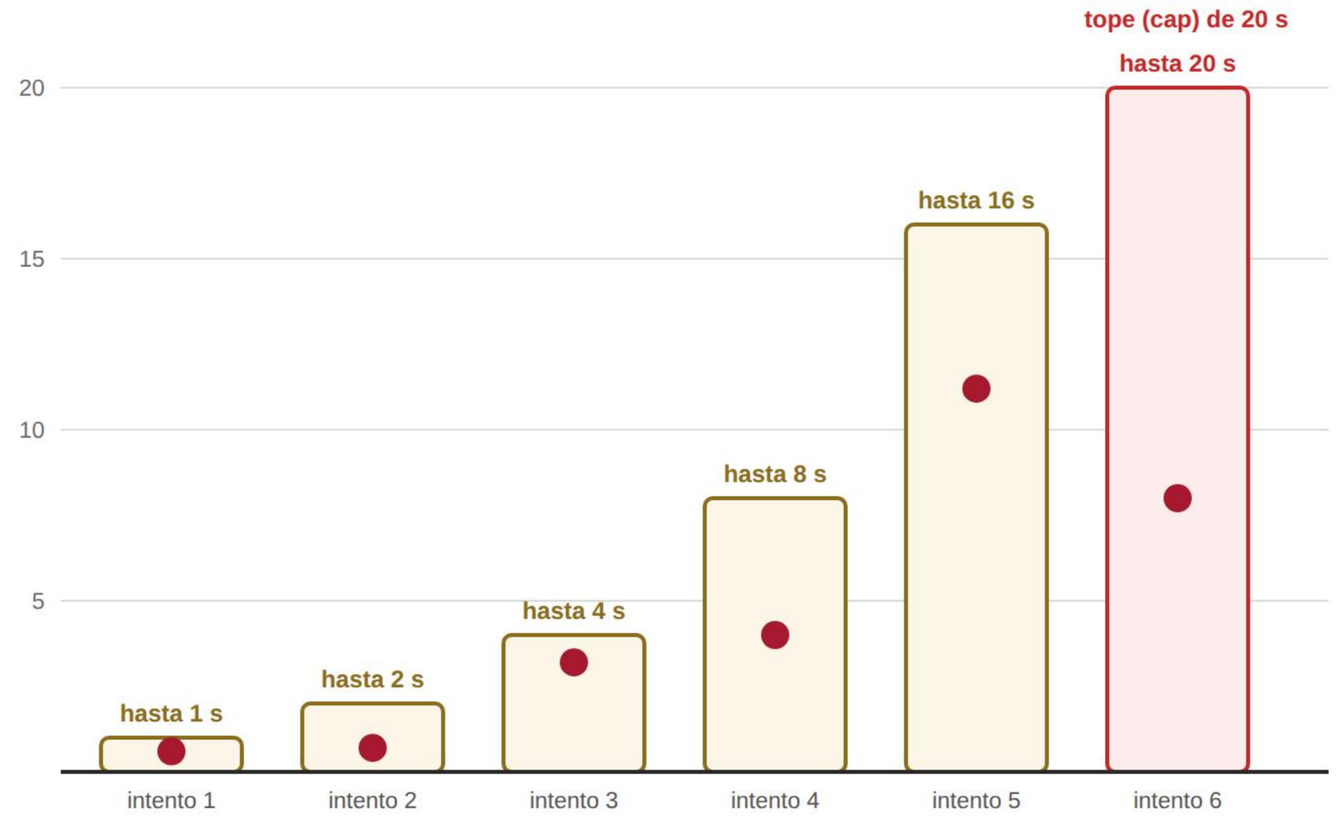
La carga queda por debajo de la capacidad desde el segundo 2 y el servicio se recupera

Fuente: adaptado de Amazon Builders Library — Timeouts, retries and backoff with jitter (cifras ilustrativas)

Retroceso exponencial con jitter

Esperar el doble en cada intento, con un tiempo aleatorio, protege al servicio que se recupera.

Espera máxima antes del siguiente intento (segundos)



● **espera real elegida al azar entre 0 y el máximo de cada barra**

Valores de ejemplo: base 1 s, factor 2, tope 20 s

Receta de reintento seguro

espera = aleatorio(0, min(tope, base × 2^intento))

1. **Exponencial** — cada intento espera el doble
2. **Jitter** — valor aleatorio: rompe la sincronía
3. **Tope** — evita esperas de minutos
4. **Máximo de intentos** — después, a la DLQ

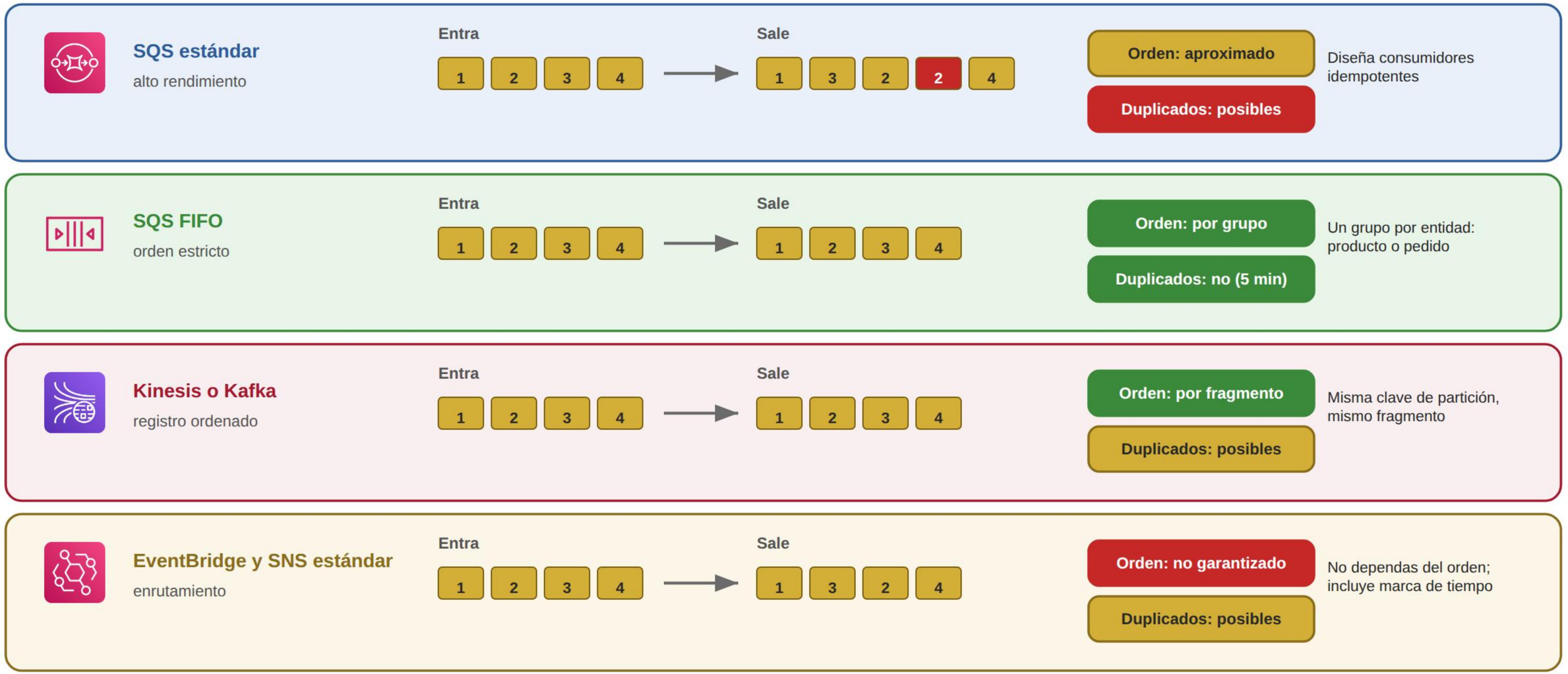
Los SDK de AWS ya reintentan con retroceso exponencial y jitter; tú decides el máximo y qué errores se reintentan

Fuente: adaptado de Amazon Builders Library — Exponential backoff and jitter (aws.amazon.com/builders-library)

ORDEN

Orden y duplicados: qué garantiza cada servicio

El orden casi siempre es por grupo o por partición, nunca global; los duplicados hay que asumirlos.



Fuente: adaptado de las guías de Amazon SQS, Kinesis Data Streams y EventBridge (docs.aws.amazon.com)

Mensajería frente a streaming: dos modelos

La cola entrega y olvida; el registro de streaming conserva y permite releer.

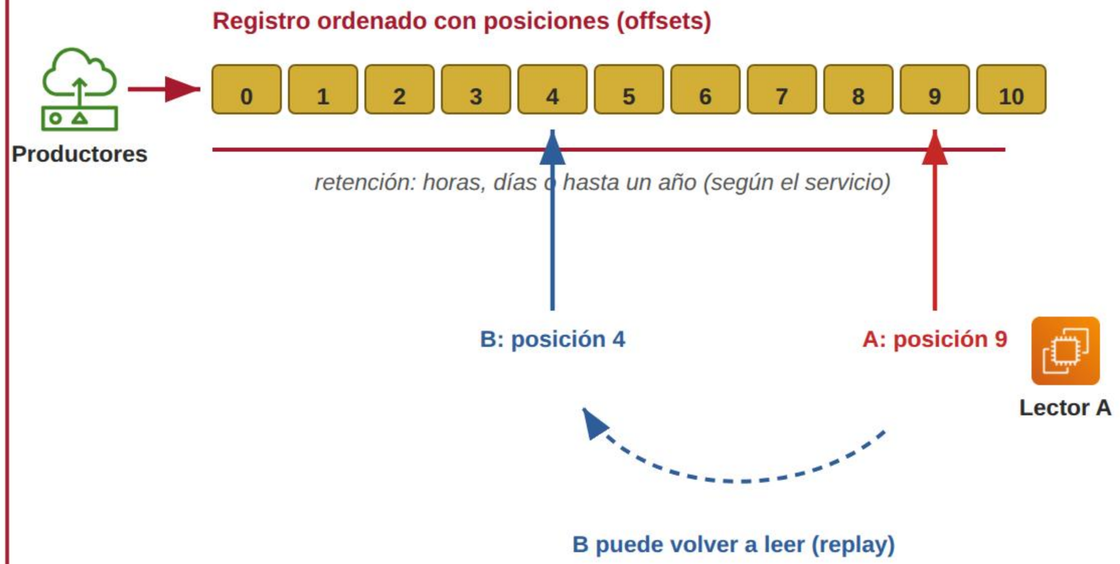
Mensajería (cola): el mensaje se consume y desaparece



1. Cada mensaje se entrega a un consumidor
2. Al confirmar, se borra de la cola
3. No se puede releer lo ya procesado

Ideal para tareas: pedidos, correos, facturas.
El trabajo se reparte y desaparece al terminar.

Streaming (registro): los datos se conservan y se releen



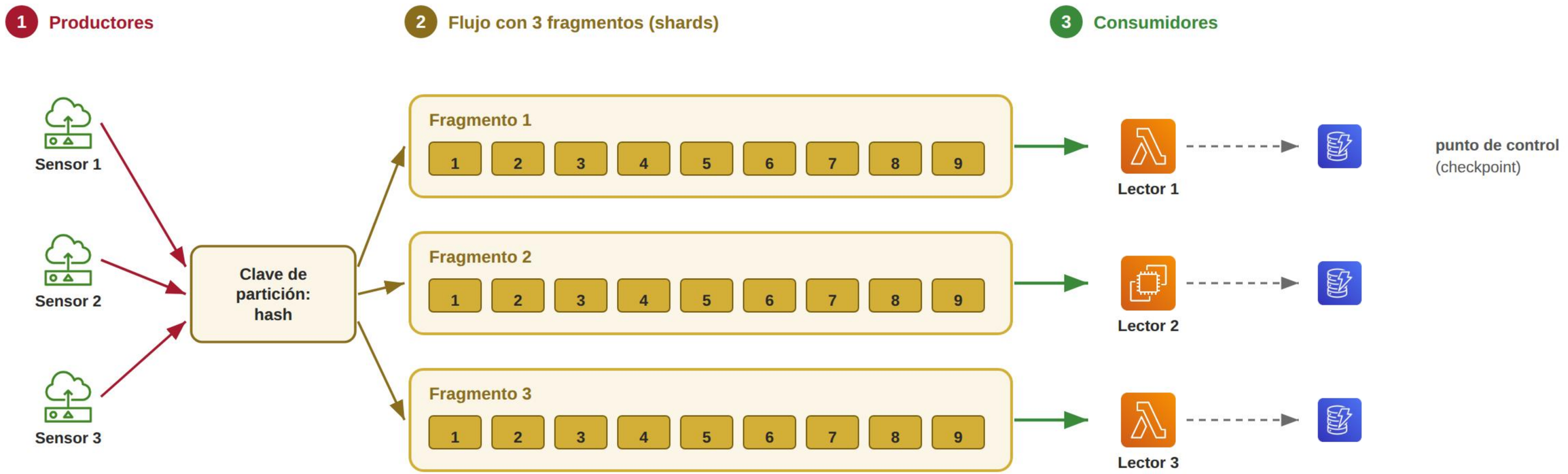
- Varios lectores independientes, cada uno con su posición
- Los registros no se borran al leerlos
- Puedes releer el pasado y reconstruir resultados

Ideal para flujos continuos: sensores, clics, registros.

Fuente: adaptado de Amazon Kinesis Data Streams Developer Guide y Apache Kafka Documentation

Kinesis y Kafka: fragmentos, particiones y lectores

La clave de partición decide el fragmento; dentro de cada fragmento el orden se conserva.



1 MB/s o 1 000 registros/s
escritura por fragmento

2 MB/s
lectura por fragmento

24 h a 365 días
retención configurable

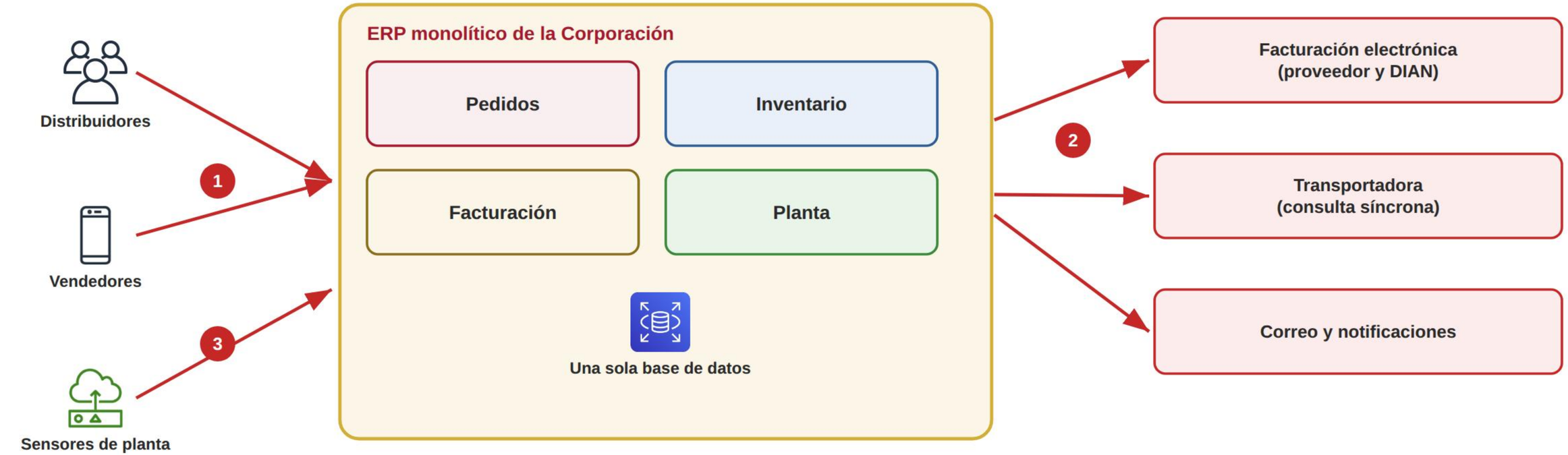
Aprovisionado o bajo demanda
modos de capacidad

Kafka (Amazon MSK): particiones y grupos de consumidores · Azure Event Hubs: particiones

Fuente: adaptado de Amazon Kinesis Data Streams Developer Guide y Amazon MSK Developer Guide (docs.aws.amazon.com)

Caso: la Corporación Industrial del Caribe hoy

Todo habla con todo de forma síncrona dentro de un ERP: cualquier pico o falla se propaga.



1 Pico al cierre de mes

La respuesta sube a 8 s y el portal da error

2 Fallas encadenadas

Si cae la facturación electrónica, se detienen los despachos

3 Lecturas perdidas

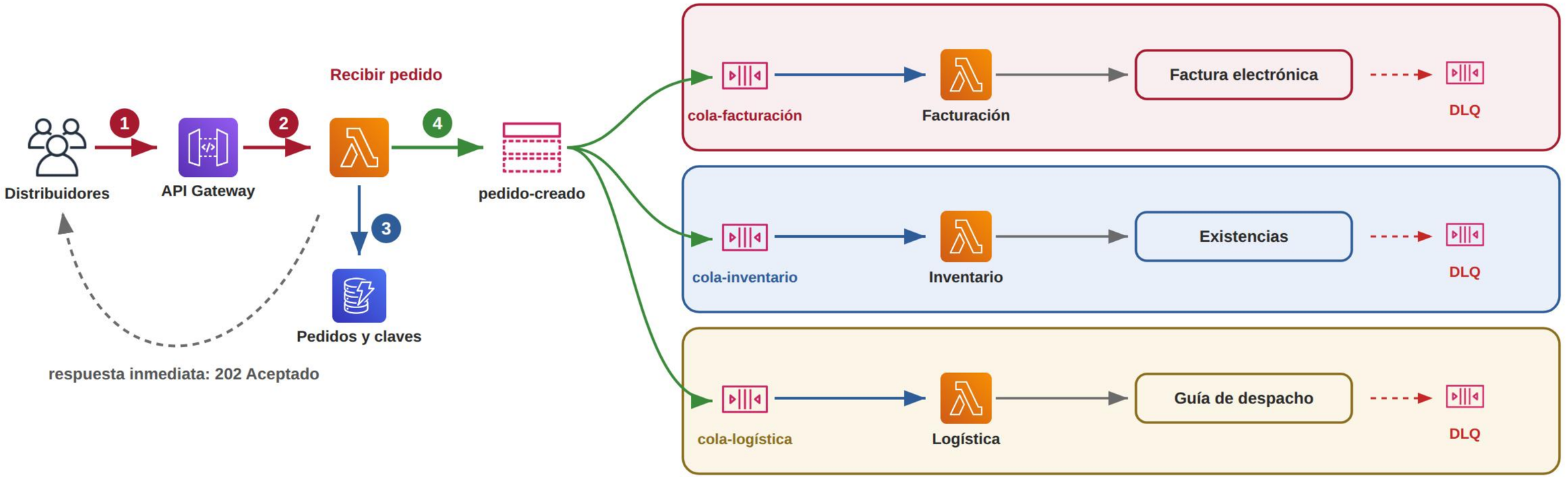
Si se reinicia el ERP, la planta pierde datos de sensores

Fuente: caso didáctico de la Corporación Industrial del Caribe (cifras ilustrativas) · íconos AWS Architecture Icons

CASO

Caso: pedidos desacoplados con SNS y SQS

El portal confirma de inmediato; facturación, inventario y logística trabajan cada uno a su ritmo.



Reintentos con retroceso exponencial y jitter

Consumidores idempotentes por número de pedido

DLQ con alarma de CloudWatch en cada cola

Escala por profundidad de la cola

Resultado: el distribuidor recibe la confirmación en menos de un segundo, aunque Facturación esté lenta o caída.

Fuente: adaptado de AWS Architecture Blog — patrón fan-out SNS y SQS (aws.amazon.com/blogs/architecture)

CASO

Caso: la planta, sensores y alarmas

Streaming para el flujo continuo de lecturas; un evento solo cuando hay algo que atender.



Por qué streaming

1. Miles de lecturas por segundo, de forma continua
2. El orden por sensor es importante
3. Releer sirve para analizar una falla pasada

Dos canales para dos necesidades

Flujo continuo y volumen alto: streaming (Kinesis) y almacenamiento histórico en S3

Hecho puntual que exige acción: evento (EventBridge) y aviso con SNS y una cola

Las lecturas normales nunca saturan a los consumidores de alarmas: solo el evento excepcional atraviesa el bus.

Umbrales y volúmenes ilustrativos.

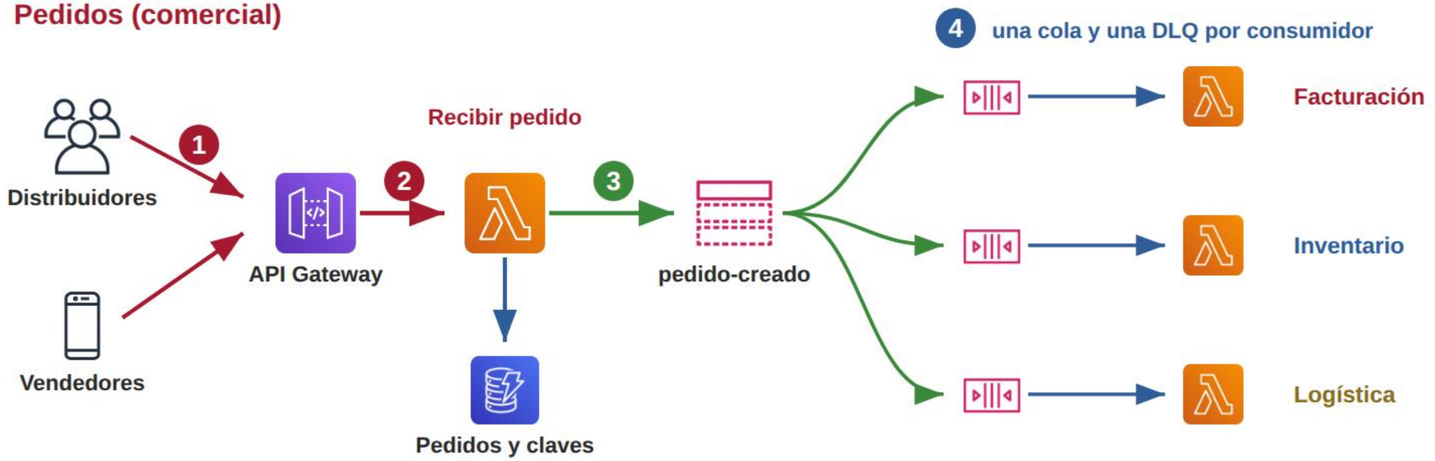
Fuente: adaptado de AWS IoT Core y Amazon Kinesis Data Streams (docs.aws.amazon.com) · íconos AWS Architecture Icons

CASO

Arquitectura desacoplada final de la Corporación

Dos flujos, pedidos y planta, con colas, temas, streaming y eventos: cada pieza falla y escala sola.

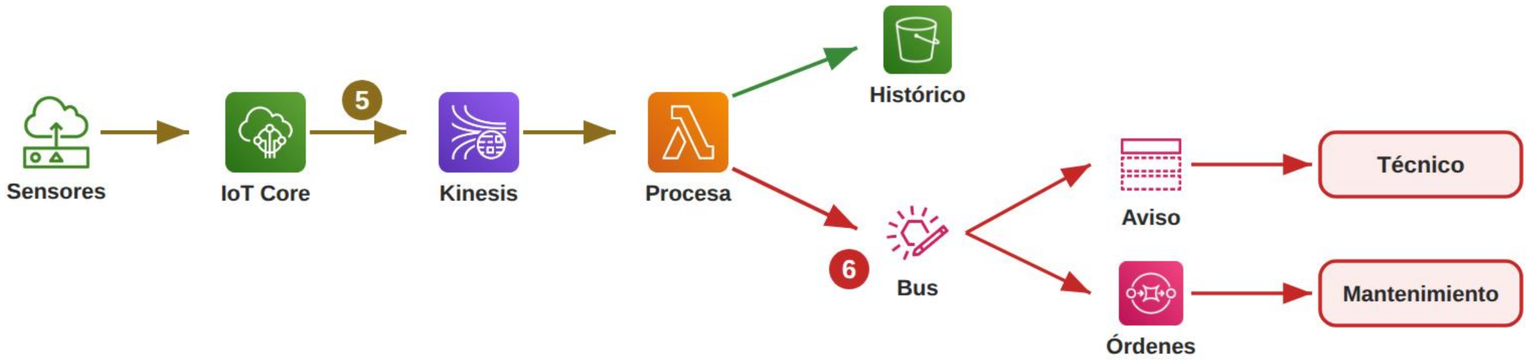
Pedidos (comercial)



Garantías de la parte comercial

- Respuesta inmediata al distribuidor
- Idempotencia por número de pedido
- Reintentos con retroceso y jitter
- Alarma de CloudWatch en cada DLQ
- Escala por profundidad de cola

Planta (operación)



Observabilidad y seguridad comunes

CloudWatch
alarmas y métricas

IAM
mínimo privilegio

Registros
trazas con id de pedido

Todo en una VPC de dos zonas de disponibilidad; los servicios administrados replican entre zonas.

Fuente: elaboración propia a partir de AWS Well-Architected — Reliability Pillar y guías de SQS, SNS, EventBridge y Kinesis

CIERRE

Semana 13 en cuatro ideas y adelanto de la Semana 14

Desacoplar con mensajes da resiliencia; hacerlo bien exige idempotencia, reintentos y vigilancia.

1 Desacopla en el tiempo



Un intermediario entre productor y consumidor da resiliencia.

Resiliencia

2 Cola, tema o bus



Reparte tareas, difunde noticias o enruta eventos por contenido.

Elige con criterio

3 Asume duplicados



Idempotencia, reintentos con retroceso y jitter, y DLQ.

Robustez

4 Mensaje o flujo



La cola borra al consumir; el streaming conserva y relee.

Dos modelos

Próxima semana: Semana 14

Serverless y microservicios

Funciones como servicio (FaaS) · gestión de APIs · Laboratorio AWS: arquitectura serverless (10 %)



Fuente: síntesis de la Semana 13 — AWS Well-Architected Framework, Reliability Pillar (docs.aws.amazon.com)